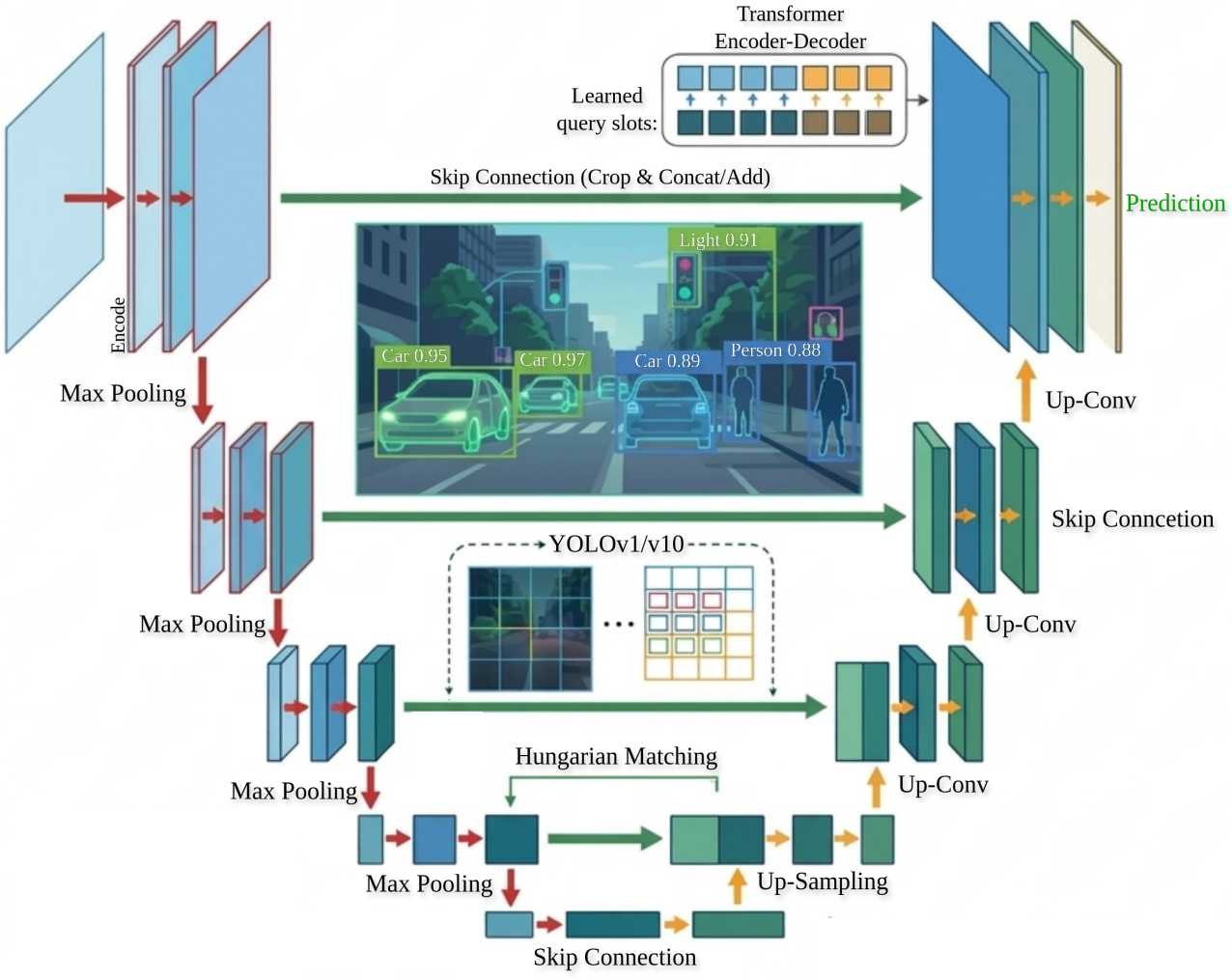
ĐỀ THI CUỐI MODULE 9

KIỂM TRA VỀ KIẾN TRÚC UNET, OBJECT DETECTION, OBJECT TRACKING & FACE RECOGNITION

(YOLO, DETR, ANCHOR-BASED, ARCFACE)

Thời gian làm bài: 210 phút



Copyright © 2026 AI VIET NAM

Mục lục

I. UNet Architecture 3

I.1. AIO25M09UNET01 - EASY . . . 3

I.2. AIO25M09UNET02 - EASY . . . 6

I.3. AIO25M09UNET03 - MEDIUM . 8

I.4. AIO25M09UNET04 - MEDIUM . 9

I.5. AIO25M09UNET05 - EASY . . . 11

I.6. AIO25M09UNET06 - MEDIUM . 12

I.7. AIO25M09UNET07 - EASY . . . 14

II. Object Detection 15

II.1. AIO25M09OD01 - EASY 15

II.2. AIO25M09OD02 - MEDIUM . . 18

II.3. AIO25M09OD03 - MEDIUM . . 22

II.4. AIO25M09OD04 - EASY 24

II.5. AIO25M09OD05 - HARD 27

II.6. AIO25M09OD06 - MEDIUM . . 29

II.7. AIO25M09OD07 - EASY 31

II.8. AIO25M09OD08 - EASY 34

II.9. AIO25M09OD09 - EASY 35

II.10. AIO25M09OD10 - EASY 36

II.11. AIO25M09OD11 - EASY 37

II.12. AIO25M09OD12 - EASY 39

III. Object Tracking 40

III.1. AIO25M09OT01 - EASY. 40

III.2. AIO25M09OT02 - EASY. 41

III.3. AIO25M09OT03 - EASY. 43

III.4. AIO25M09OT04 - MEDIUM. . . 44

IV. Face Recognition 46

IV.1. AIO25M09FACE01 - EASY . . . 46

IV.2. AIO25M09FACE02 - EASY . . . 48

IV.3. AIO25M09FACE03 - MEDIUM . 50

IV.4. AIO25M09FACE04 - MEDIUM . 52

V. YOLOv1 54

V.1. AIO25M09YOLO01 - EASY . . . 54

V.2. AIO25M09YOLO02 - EASY . . . 56

V.3. AIO25M09YOLO03 - EASY . . . 58

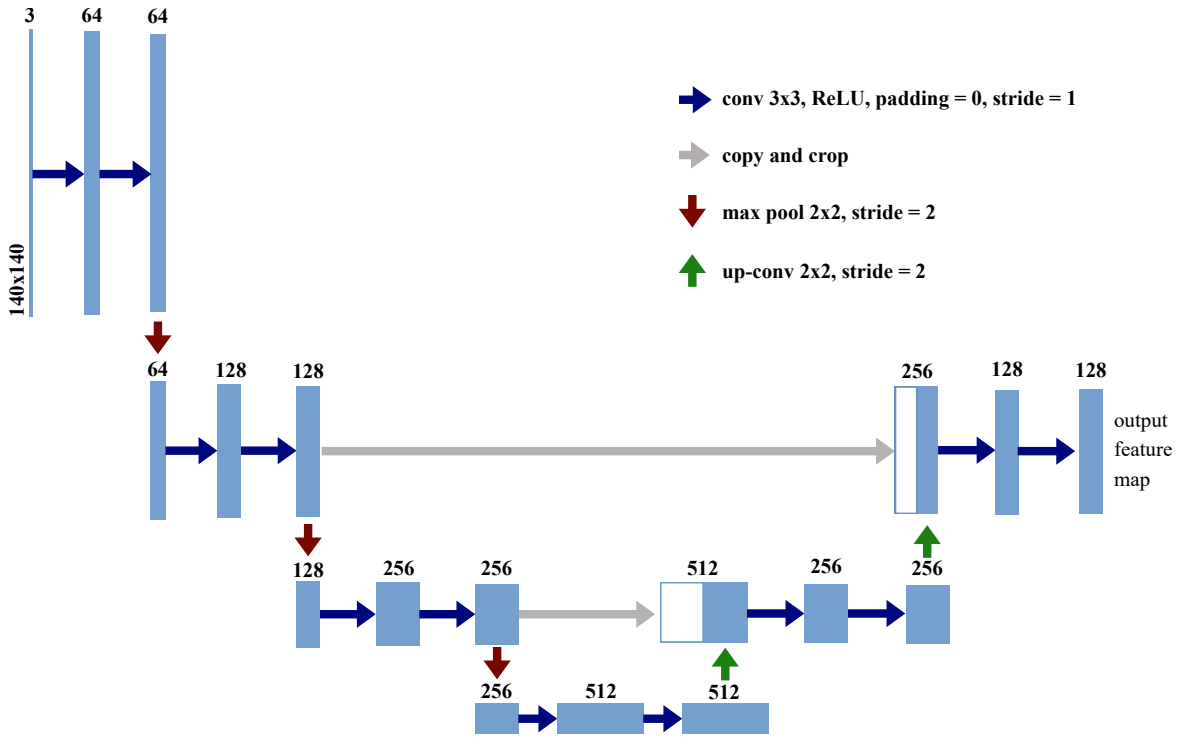
V.4. AIO25M09YOLO04 - MEDIUM . 61

V.5. AIO25M09YOLO05 - MEDIUM . 64

I. UNet Architecture

I.1. AIO25M09UNET01 - EASY

Skip Connection là cơ chế cốt lõi của kiến trúc U-Net, đóng vai trò ghép nối các đặc trưng từ nhánh Encoder sang nhánh Decoder. Tuy nhiên, việc không sử dụng kỹ thuật padding sẽ khiến kích thước feature map bị thu hẹp dần qua từng lớp, dẫn đến hiện tượng tensor ở nhánh Encoder luôn lớn hơn so với nhánh Decoder. Để khắc phục điều này, thao tác center crop cần được thực hiện chuẩn xác nhằm đảm bảo hai tensor hoàn toàn khớp nhau trước khi tiến hành phép nối.



Dựa trên sơ đồ kiến trúc được mô tả như trong hình, hãy xác định kích thước $[C, H, W]$ của output feature map cuối cùng là bao nhiêu?

A. $[128, 32, 32]$.

B. $[128, 28, 28]$.

C. $[128, 100, 100]$.

D. $[128, 52, 52]$.

Đáp án: B

Giải thích: Colab Notebook

Quy ước công thức biến đổi không gian (áp dụng cho chiều cao H, chiều rộng W tính tương tự):

- **Lớp Tích chập - Conv2D** (Kernel $K = 3$, Padding $P = 0$, Stride $S = 1$):
$$H_{out} = \left\lfloor \frac{H_{in} - K + 2P}{S} \right\rfloor + 1 = \left\lfloor \frac{H_{in} - 3 + 0}{1} \right\rfloor + 1 = H_{in} - 2$$
- **MaxPool2D** (Kernel $K = 2$, Padding $P = 0$, Stride $S = 2$):
$$H_{out} = \left\lfloor \frac{H_{in} - K + 2P}{S} \right\rfloor + 1 = \left\lfloor \frac{H_{in} - 2 + 0}{2} \right\rfloor + 1 = \frac{H_{in}}{2}$$
- **UpConv2D** (Kernel $K = 2$, Padding $P = 0$, Stride $S = 2$):
$$H_{out} = (H_{in} - 1) \times S - 2P + K = (H_{in} - 1) \times 2 - 0 + 2 = 2H_{in}$$

1. Nhánh đi xuống (Encoder)

Đầu vào (Input): $3 \times 140 \times 140$

Encoder Block 1:

$$\begin{aligned} \text{Qua Conv 1 (64 kênh):} \quad & 140 - 2 = 138 \quad \Rightarrow \mathbf{64 \times 138 \times 138} \\ \text{Qua Conv 2 (64 kênh):} \quad & 138 - 2 = 136 \quad \Rightarrow \mathbf{64 \times 136 \times 136} \\ \text{Qua MaxPool2D:} \quad & \frac{136}{2} = 68 \quad \Rightarrow \mathbf{64 \times 68 \times 68} \end{aligned}$$

Encoder Block 2:

$$\begin{aligned} \text{Qua Conv 1 (128 kênh):} \quad & 68 - 2 = 66 \quad \Rightarrow \mathbf{128 \times 66 \times 66} \\ \text{Qua Conv 2 (128 kênh):} \quad & 66 - 2 = 64 \quad \Rightarrow \mathbf{128 \times 64 \times 64} \end{aligned}$$

$\rightarrow$ *Trích xuất tensor này làm Skip Connection 1:* $\mathbf{128 \times 64 \times 64}$

$$\text{Qua MaxPool2D:} \quad \frac{64}{2} = 32 \quad \Rightarrow \mathbf{128 \times 32 \times 32}$$

Encoder Block 3:

$$\begin{aligned} \text{Qua Conv 1 (256 kênh):} \quad & 32 - 2 = 30 \quad \Rightarrow \mathbf{256 \times 30 \times 30} \\ \text{Qua Conv 2 (256 kênh):} \quad & 30 - 2 = 28 \quad \Rightarrow \mathbf{256 \times 28 \times 28} \end{aligned}$$

$\rightarrow$ *Trích xuất tensor này làm Skip Connection 2:* $\mathbf{256 \times 28 \times 28}$

$$\text{Qua MaxPool2D:} \quad \frac{28}{2} = 14 \quad \Rightarrow \mathbf{256 \times 14 \times 14}$$

2. Bottleneck (Đáy mạng)

$$\begin{aligned} \text{Qua Conv 1 (512 kênh):} \quad & 14 - 2 = 12 \quad \Rightarrow \mathbf{512 \times 12 \times 12} \\ \text{Qua Conv 2 (512 kênh):} \quad & 12 - 2 = 10 \quad \Rightarrow \mathbf{512 \times 10 \times 10} \end{aligned}$$

3. Nhánh đi lên (Decoder) và Output

Decoder Block 1 (Tương ứng Encoder 3):

- **Up-conv (256 kênh):** $10 \times 2 = 20 \Rightarrow \mathbf{256 \times 20 \times 20}$
- **Center Crop Skip Connection 2:** Cắt từ 28×28 xuống 20×20 ($\Delta = 8$, cắt 4 pixel mỗi cạnh).
- **Concatenate:** $256 + 256 = 512$ kênh $\Rightarrow \mathbf{512 \times 20 \times 20}$

$$\begin{aligned} &\text{Qua Conv 1 (256 kênh):} \quad 20 - 2 = 18 \quad \Rightarrow \mathbf{256 \times 18 \times 18} \\ &\text{Qua Conv 2 (256 kênh):} \quad 18 - 2 = 16 \quad \Rightarrow \mathbf{256 \times 16 \times 16} \end{aligned}$$

Decoder Block 2 (Tương ứng Encoder 2):

- **Up-conv (128 kênh):** $16 \times 2 = 32 \Rightarrow \mathbf{128 \times 32 \times 32}$
- **Center Crop Skip Connection 1:** Cắt từ 64×64 xuống 32×32 ($\Delta = 32$, cắt 16 pixel mỗi cạnh).
- **Concatenate:** $128 + 128 = 256$ kênh $\Rightarrow \mathbf{256 \times 32 \times 32}$

Tính toán Output cuối cùng:

$$\begin{aligned} &\text{Qua Conv 1 (128 kênh):} \quad 32 - 2 = 30 \quad \Rightarrow \mathbf{128 \times 30 \times 30} \\ &\text{Qua Conv 2 (128 kênh):} \quad 30 - 2 = 28 \quad \Rightarrow \mathbf{128 \times 28 \times 28} \end{aligned}$$

Kết luận: Kích thước của Output Feature Map cuối cùng là $\mathbf{128 \times 28 \times 28}$.

I.2. AIO25M09UNET02 - EASY

Xét phần **Encoder** của một mô hình U-Net được mô tả bởi đoạn code PyTorch dưới đây. Encoder gồm 3 encoder blocks liên tiếp, mỗi block gồm hai lớp `Conv2d(3×3)` theo sau bởi `BatchNorm2d` và `ReLU`.

U-Net Encoder

```
encoder = nn.Sequential(

    # ----- Encoder Block 1 -----
    nn.Conv2d(1, 64, kernel_size=3, bias=False),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    nn.Conv2d(64, 64, kernel_size=3, bias=False),
    nn.BatchNorm2d(64),
    nn.ReLU(),

    # ----- Encoder Block 2 -----
    nn.Conv2d(64, 128, kernel_size=3, bias=False),
    nn.BatchNorm2d(128),
    nn.ReLU(),
    nn.Conv2d(128, 128, kernel_size=3, bias=False),
    nn.BatchNorm2d(128),
    nn.ReLU(),

    # ----- Encoder Block 3 -----
    nn.Conv2d(128, 256, kernel_size=3, bias=False),
    nn.BatchNorm2d(256),
    nn.ReLU(),
    nn.Conv2d(256, 256, kernel_size=3, bias=False),
    nn.BatchNorm2d(256),
    nn.ReLU()
)
```

Dựa trên kiến trúc Encoder được mô tả trong đoạn code trên, hỏi tổng số lượng tham số huấn luyện được (trainable parameters) của toàn bộ Encoder là bao nhiêu?

A. 1145152.

B. 1144128.

C. 1147200.

D. 1150464.

Đáp án: A

Giải thích: Colab Notebook

Bài toán yêu cầu tính **tổng số lượng tham số huấn luyện được (trainable parameters)** của toàn bộ Encoder. Các tham số này đến từ hai loại lớp chính là `Conv2d` và `BatchNorm2d`.

- **Conv2d**: số tham số được tính theo công thức

$$\text{Params} = C_{out} \times C_{in} \times K \times K$$

  Trong đó $K = 3$ và `bias=False`, nên mỗi lớp convolution không có tham số bias.

- **BatchNorm2d**: mỗi kênh C có hai tham số trainable là γ (scale) và β (shift), do đó

$$\text{Params} = 2 \times C$$

Encoder Block 1

$$\begin{aligned}
\text{Conv}(1 \to 64): &\ 64 \times 1 \times 3 \times 3 = 576 \\
\text{BatchNorm}(64): &\ 2 \times 64 = 128 \\
\text{Conv}(64 \to 64): &\ 64 \times 64 \times 3 \times 3 = 36864 \\
\text{BatchNorm}(64): &\ 2 \times 64 = 128
\end{aligned}$$

$$\text{Tổng Block 1} = 37696$$

Encoder Block 2

$$\begin{aligned}
\text{Conv}(64 \to 128): &\ 128 \times 64 \times 3 \times 3 = 73728 \\
\text{BatchNorm}(128): &\ 2 \times 128 = 256 \\
\text{Conv}(128 \to 128): &\ 128 \times 128 \times 3 \times 3 = 147456 \\
\text{BatchNorm}(128): &\ 2 \times 128 = 256
\end{aligned}$$

$$\text{Tổng Block 2} = 221696$$

Encoder Block 3

$$\begin{aligned}
\text{Conv}(128 \to 256): &\ 256 \times 128 \times 3 \times 3 = 294912 \\
\text{BatchNorm}(256): &\ 2 \times 256 = 512 \\
\text{Conv}(256 \to 256): &\ 256 \times 256 \times 3 \times 3 = 589824 \\
\text{BatchNorm}(256): &\ 2 \times 256 = 512
\end{aligned}$$

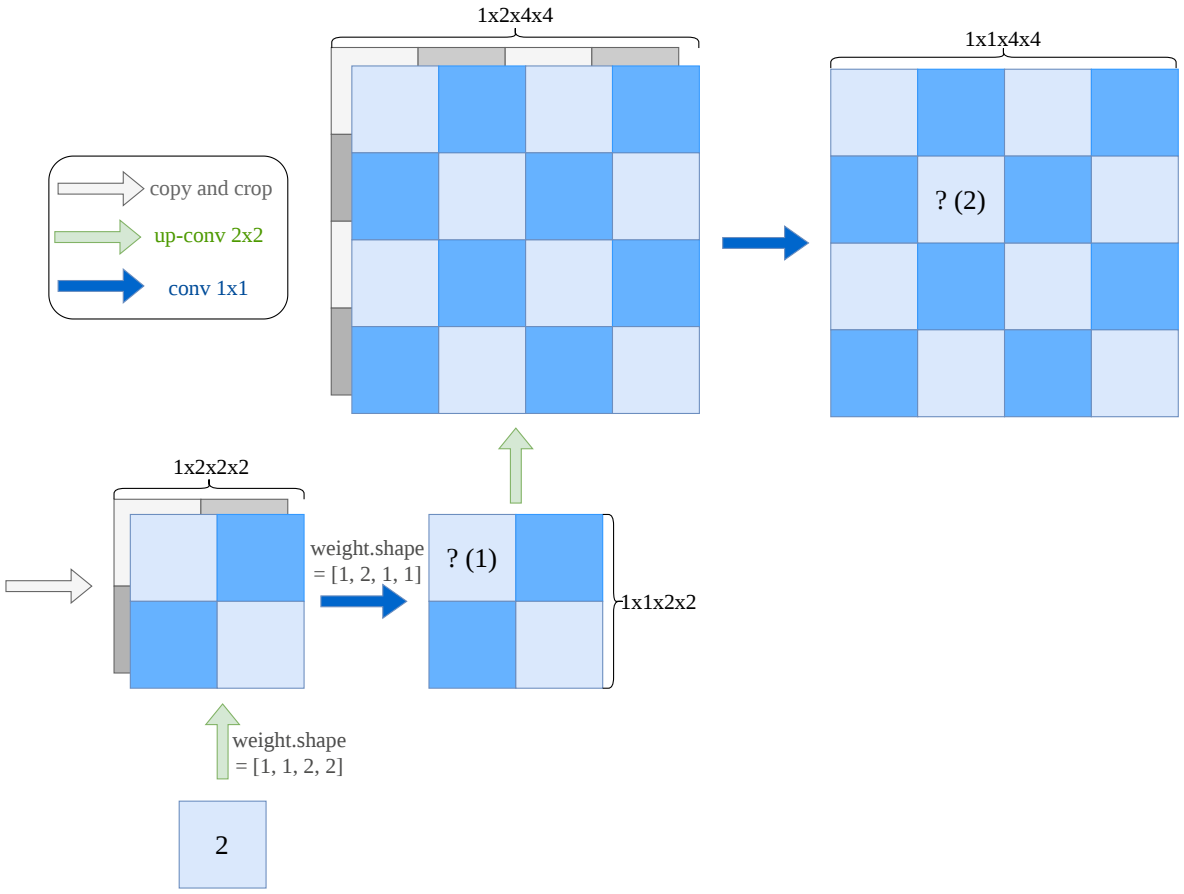
$$\text{Tổng Block 3} = 885760$$

Tổng số tham số huấn luyện được của toàn bộ Encoder

$$37696 + 221696 + 885760 = 1145152$$

I.3. AIO25M09UNET03 - MEDIUM

Dựa trên sơ đồ kiến trúc dưới đây, bạn hãy hoàn thiện các phần code còn thiếu cho block Decoder và mô hình TwoStageDecoder tại Colab Notebook.



Để mô hình thực hiện đúng quy trình, xin lưu ý rằng cấu trúc này bao gồm 2 stage Decoder liên tiếp hoàn toàn không sử dụng tham số `bias`. Ngoài ra, trọng số của lớp `convblock` cần được gán chính xác theo thứ tự của các kênh đầu vào (input channels). Bạn có thể tham khảo thêm gợi ý chi tiết tại mục `Set_fixed_weights` trong notebook.

Sau khi hoàn thiện đoạn code và thực hiện quá trình lan truyền xuôi (forward pass), giá trị tại hai vị trí ?(1) và ?(2) trên sơ đồ lần lượt là bao nhiêu?

(Lưu ý kết quả được làm tròn đến 4 chữ số thập phân.)

A. $(2.6744, -0.4609)$.

B. $(4.5765, 3.8665)$.

C. $(-0.1248, 2.8253)$.

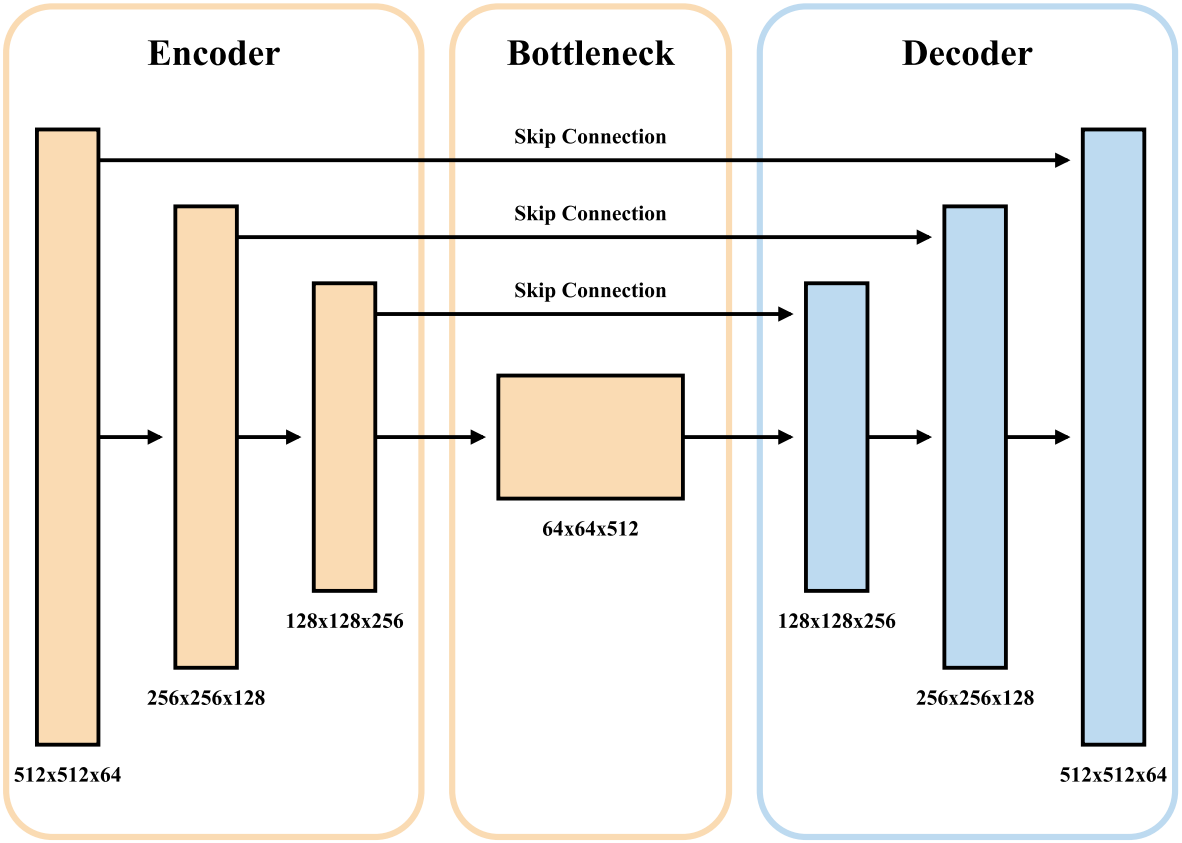
D. $(4.1406, 4.0682)$.

Đáp án: A

Giải thích: Colab Notebook

I.4. AIO25M09UNET04 - MEDIUM

Xét encoder của kiến trúc U-Net như minh họa trong hình dưới đây. Ảnh đầu vào có kích thước $512 \times 512 \times 1$. Kích thước các feature map của encoder và bottleneck được ghi trực tiếp trong hình.



Trong quá trình training, tất cả activation của encoder và input phải được lưu lại để phục vụ cho backpropagation và skip connections trong decoder. Giả sử các tensor sử dụng BF16 (2 bytes / phần tử). Memory của một tensor có thể tính theo công thức:

$$Memory = \frac{B \times H \times W \times C \times 2 \text{ bytes}}{1024^2} \text{ MB}$$

Trong đó, B là batch size, H, W là chiều cao và chiều rộng của feature map và C là số kênh. Với GPU có 4GB VRAM, batch size lớn nhất có thể sử dụng để train mô hình (trong các lựa chọn sau) là bao nhiêu? (Bỏ qua memory của weight và các overhead khác.)

A. 32.

B. 64.

C. 128.

D. 256.

Đáp án: B.

Giải thích: Colab Notebook

Bộ nhớ activation được tính:

$$Memory = \frac{B \times H \times W \times C \times 2 \text{ bytes}}{1024^2} \text{ MB}$$

Ta tính bộ nhớ của các activation tensors (bao gồm input và các feature map):

$$512 \times 512 \times 1 \times 2 \text{ bytes} = 524{,}288 \text{ bytes} \approx 0.5 \text{ MB} \quad \text{(Input)}$$

$$512 \times 512 \times 64 \times 2 \text{ bytes} = 33{,}554{,}432 \text{ bytes} \approx 32 \text{ MB} \quad \text{(Encoder Block 1)}$$

$$256 \times 256 \times 128 \times 2 \text{ bytes} = 16{,}777{,}216 \text{ bytes} \approx 16 \text{ MB} \quad \text{(Encoder Block 2)}$$

$$128 \times 128 \times 256 \times 2 \text{ bytes} = 8{,}388{,}608 \text{ bytes} \approx 8 \text{ MB} \quad \text{(Encoder Block 3)}$$

$$64 \times 64 \times 512 \times 2 \text{ bytes} = 4{,}194{,}304 \text{ bytes} \approx 4 \text{ MB} \quad \text{(Bottleneck)}$$

Tổng bộ nhớ activation:

$$M_{total} = B(0.5 + 32 + 16 + 8 + 4) = 60.5B \text{ MB} \quad (1)$$

GPU có:

$$4 \text{ GB} = 4096 \text{ MB} \quad (2)$$

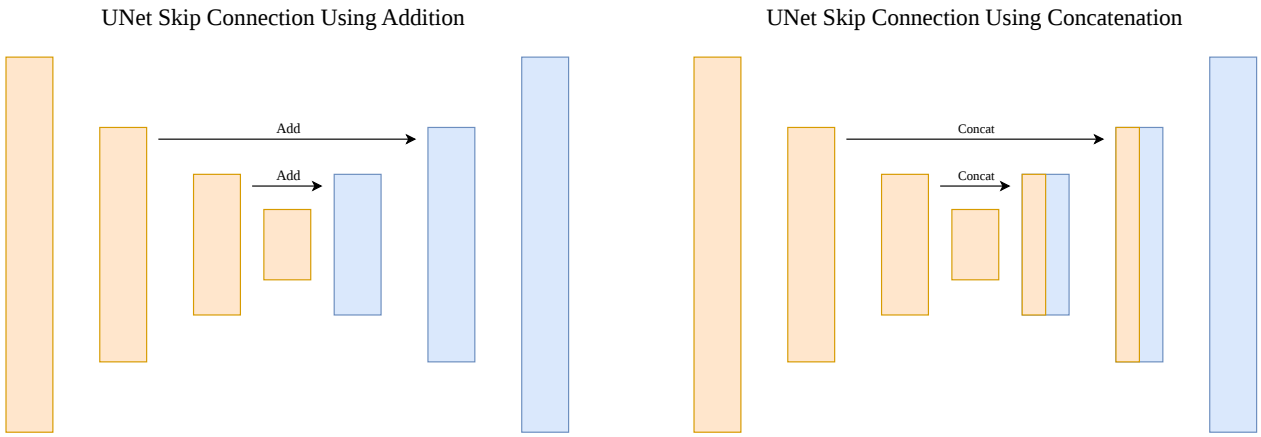
$$(1), (2) \Rightarrow 60.5B \leq 4096 \Leftrightarrow B \leq \frac{4096}{60.5} \approx 67$$

Do đó, batch size tối đa xấp xỉ 67. Trong các lựa chọn đã cho, giá trị lớn nhất không vượt quá giới hạn này là:

$$B = 64$$

I.5. AIO25M09UNET05 - EASY

Trong kiến trúc U-Net, các skip connection tại khối decoder thường sử dụng phép `concatenate`. Tuy nhiên, một biến thể phổ biến khác là sử dụng phép `add`.



Giả sử sau bước up-convolution, tensor từ decoder và tensor từ encoder đã đồng nhất về mặt kích thước không gian lẫn số lượng kênh, đủ điều kiện để thực hiện trực tiếp phép `add`, bạn hãy hoàn thiện các đoạn code còn thiếu trong Colab Notebook đính kèm để block Decoder có thể hỗ trợ cả hai chế độ `concat` và `add`.

(Lưu ý rằng tất cả các lớp convolution trong bài đều được thiết lập `bias=False`.)

Sau khi hoàn thiện và thực thi đoạn code trên, số lượng phần tử của tham số `conv1.weight` trong hai trường hợp sử dụng block `concat` và block `add` lần lượt là bao nhiêu?

A. (9216, 9216).

B. (18432, 9216).

C. (18432, 18432).

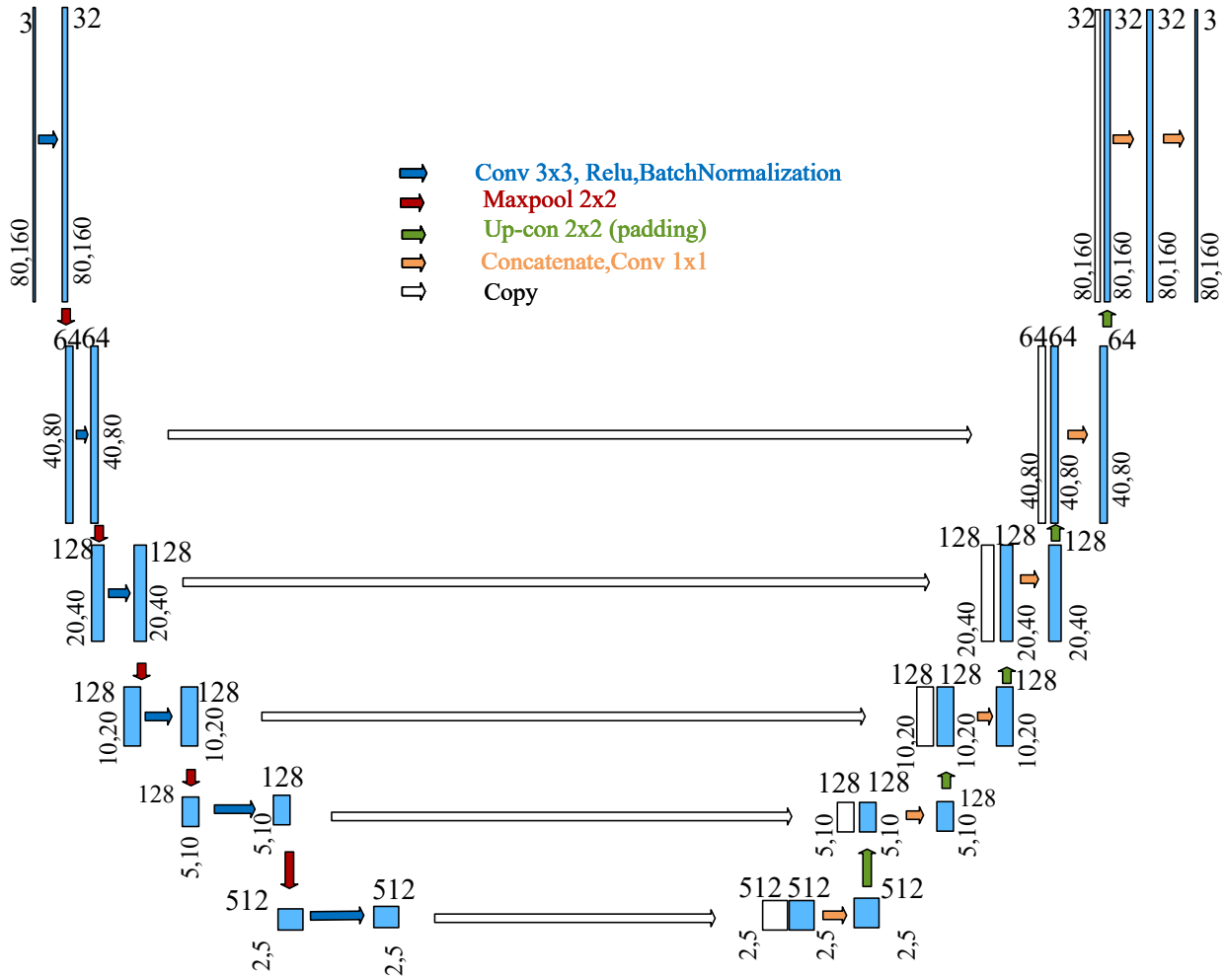
D. (4096, 2048).

Đáp án: B

Giải thích: Colab Notebook

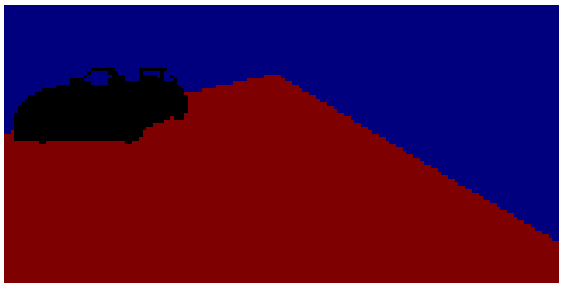
I.6. AIO25M09UNET06 - MEDIUM

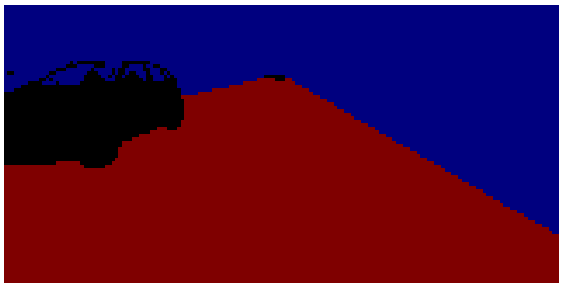
Xét bài toán phân vùng làn đường (Lane Segmentation) trên tập dữ liệu ảnh giao thông. Mô hình sẽ nhận đầu vào là một ảnh RGB có kích thước 80×160 và thực hiện dự đoán một mặt nạ phân vùng (segmentation mask) gồm 3 lớp: lớp nền (background), làn đường bên trái và làn đường bên phải.



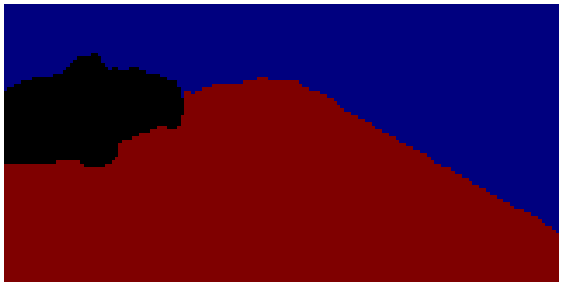
Bạn hãy hoàn thiện code tại Colab Notebook, và huấn luyện mô hình. sau khi hoàn tất quá trình huấn luyện, hãy sử dụng mô hình này để dự báo cho hình ảnh `Town01_0001_000552.png` nằm trong thư mục `Val` của tập dữ liệu.

Trong các lựa chọn dưới đây, đâu là kết quả phân vùng chính xác do mô hình trả về?

A. 

C. 

B. 

D. 

Đáp án: B.

Giải thích: Colab Notebook

I.7. AIO25M09UNET07 - EASY

Trong bài toán phân vùng ảnh (image segmentation), đầu ra của mô hình thường là một bản đồ xác suất (probability map). Giả sử ta thực hiện chuyển đổi probability map này thành mặt nạ nhị phân (binary mask) dựa trên quy tắc ngưỡng như sau:

$$\hat{Y}_{i,j} = \begin{cases} 1 & \text{nếu } p_{i,j} \geq 0.5, \\ 0 & \text{nếu } p_{i,j} < 0.5. \end{cases}$$

Cho mặt nạ nhãn chuẩn (ground truth mask) Y và bản đồ xác suất P được mô tả trong hình dưới đây:

$Y =$

1	1	0	0
1	0	0	0
0	1	1	0
0	0	1	1

$P =$

0.90	0.70	0.40	0.20
0.49	0.60	0.30	0.10
0.20	0.90	0.55	0.45
0.10	0.20	0.51	0.80

Biết các công thức tính hệ số Dice và chỉ số IoU (Intersection over Union) được định nghĩa lần lượt là:

$$Dice = \frac{2TP}{2TP + FP + FN}, \qquad IoU = \frac{TP}{TP + FP + FN}.$$

Hãy tính giá trị Dice và IoU của dự đoán P so với mặt nạ nhãn chuẩn Y sau khi áp dụng bước thresholding. (Lưu ý kết quả được làm tròn đến 4 chữ số thập phân.)

A. (0.7500, 0.6000).

B. (0.8571, 0.7500).

C. (0.8750, 0.7778).

D. (0.9333, 0.8750).

Đáp án: B

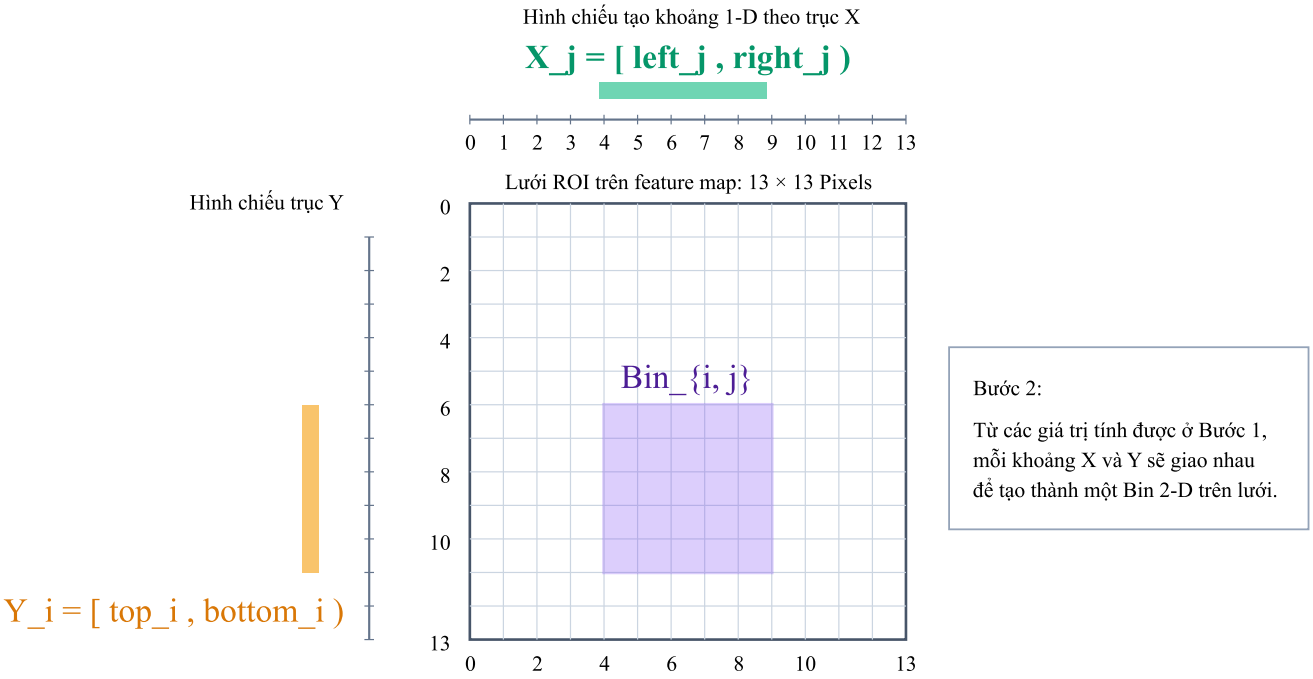
Giải thích: Colab Notebook

II. Object Detection

II.1. AIO25M09OD01 - EASY

Trong Object Detection, **Region of Interest (ROI)** hiểu đơn giản là các vùng ảnh có khả năng chứa đối tượng. Vì vật thể trong ảnh có kích thước khác nhau, các ROI này khi chiếu lên feature map cũng có độ lớn hoàn toàn khác nhau. Tuy nhiên, các tầng phía sau của mô hình thường yêu cầu đầu vào có kích thước cố định. Để đáp ứng điều này, **Spatial Pyramid Pooling (SPP)** chia mỗi ROI thành một lưới gồm nhiều ô nhỏ (gọi là các **bin**) với số lượng định trước. Bằng cách trích xuất đặc trưng (pooling) trên từng bin, ta sẽ luôn thu được kết quả đầu ra đồng nhất, bất kể ROI ban đầu to hay nhỏ.

Câu hỏi này chỉ xét bước **xác định cách chia ROI thành các bin**. Cụ thể, với một ROI kích thước 13×13 trên feature map, ta muốn chia nó thành lưới 6×6 (36 bin). Vì 13 không chia hết cho 6, ranh giới của mỗi bin phải được xác định bằng phép lượng tử hóa tọa độ.



Trên hình, các dải màu theo trục X biểu diễn các khoảng X_j, còn mỗi bin 2-D được tạo bằng cách kết hợp một khoảng theo hàng với một khoảng theo cột. Để xác định các khoảng này, ta thực hiện lần lượt 2 bước sau:

- **Bước 1:** xác định biên trái và biên phải của từng bin theo công thức lượng tử hóa:

$$\text{left}_j = \left\lfloor \frac{j \cdot 13}{6} \right\rfloor, \qquad \text{right}_j = \left\lceil \frac{(j+1) \cdot 13}{6} \right\rceil$$

  với $j = 0, \ldots, 5$.

- **Bước 2:** từ các biên này, suy ra các interval 1-D theo quy ước:

$$X_j = [\text{left}_j,\ \text{right}_j)$$

Lưu ý: Trong bài này, các interval đều được hiểu theo quy ước nửa kín $[left,\ right)$.

Câu hỏi: Dãy interval 1-D theo cột (X_j) và nhận xét đi kèm nào sau đây là **đúng**?

A. X_0=$[0, 2)$, X_1=$[2, 4)$, X_2=$[4, 7)$, X_3=$[7, 9)$, X_4=$[9, 11)$, X_5=$[11, 13)$
Mỗi bin gồm 2 phần tử, không chồng lấn.

B. X_0=$(0, 3]$, X_1=$(2, 5]$, X_2=$(4, 7]$, X_3=$(6, 9]$, X_4=$(8, 11]$, X_5=$(10, 13]$
Mỗi bin gồm 2 phần tử, chồng lấn.

C. X_0=$[0, 3)$, X_1=$[2, 5)$, X_2=$[4, 7)$, X_3=$[6, 9)$, X_4=$[8, 11)$, X_5=$[10, 13)$
Mỗi bin gồm 3 phần tử, chồng lấn.

D. X_0=$(0, 3]$, X_1=$(3, 5]$, X_2=$(5, 7]$, X_3=$(7, 9]$, X_4=$(9, 11]$, X_5=$(11, 13]$
Mỗi bin gồm 3 phần tử, không chồng lấn.

Đáp án: C

Bước 1: Tính biên của các bin theo công thức lượng tử hóa
Với $j = 0, \ldots, 5$,

$$\text{left}_j = \left\lfloor \frac{j \cdot 13}{6} \right\rfloor, \qquad \text{right}_j = \left\lceil \frac{(j+1) \cdot 13}{6} \right\rceil.$$

j	$j \cdot 13/6$	$(j+1) \cdot 13/6$	left_j	right_j
0	0	2.167	0	3
1	2.167	4.333	2	5
2	4.333	6.5	4	7
3	6.5	8.667	6	9
4	8.667	10.833	8	11
5	10.833	13	10	13

Bước 2: Suy ra các interval
Theo quy ước nửa kín $[left, right)$, ta có

$$X_0 = [0, 3),\ X_1 = [2, 5),\ X_2 = [4, 7),\ X_3 = [6, 9),\ X_4 = [8, 11),\ X_5 = [10, 13).$$

Tương tự cho hàng:

$$Y_i = [\text{top}_i, \text{bottom}_i).$$

Nhận xét về overlap: mỗi bin có độ rộng 3, nên tổng độ rộng là $6 \times 3 = 18 > 13$. Vì vậy các bin kề nhau **bắt buộc chồng lấn**. Cụ thể:

$$X_0 \cap X_1 = [2, 3), \quad X_1 \cap X_2 = [4, 5), \quad \ldots$$

Vậy phương án đúng là

$$X_0 = [0, 3),\ X_1 = [2, 5),\ \ldots,\ X_5 = [10, 13),$$

mỗi bin gồm 3 phần tử và **có chồng lấn**.

II.2. AIO25M09OD02 - MEDIUM

Trong YOLOv1, mỗi cell dự đoán nhiều bounding box. Mỗi bounding box có một confidence riêng, còn các xác suất lớp được dự đoán chung cho toàn bộ cell. Xét hai cell kề nhau là **Cell A** và **Cell B**, mỗi cell dự đoán 2 box cho ba lớp `Person`, `Car`, `Dog`. Biết rằng ground-truth object là một **Car** và **tâm của ground-truth nằm trong Cell A**.

Ta thực hiện lần lượt 3 bước sau:

1. tính IoU giữa từng predicted box và ground-truth;
2. ở training, chọn box có IoU lớn nhất trong cell chứa tâm object làm *responsible predictor*;
3. ở test time, tính *car score* của từng box để xác định box có điểm cao nhất trước khi áp dụng NMS.

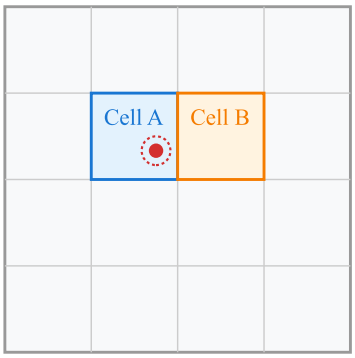
Biết rằng:

$$\text{Confidence} = Pr(\text{Object}) \times \text{IoU}$$

và ở test time, điểm cho lớp `Car` của mỗi box được tính bởi:

$$P(\text{Car} \mid \text{Object}) \times \text{Confidence}_{\text{box}}$$

Cell chứa tâm GT



● Tâm GT (Car)

Tâm GT ∈ Cell A

⇒ ở training, chỉ có A1 và A2 cạnh tranh để trở thành responsible predictor.

Kết quả dự đoán của mạng

Dùng cho Bước (1): tính IoU giữa từng predicted box và GT.

Công thức: IoU = Area(inter) / Area(union)

Cell A	P(Car) = 0.70 | P(Person) = 0.10 | P(Dog) = 0.20		
Box	Confidence (C)	Area(inter)	Area(union)
A1	0.55	0.086	0.150
A2	0.50	0.126	0.164

Cell B	P(Car) = 0.60 | P(Person) = 0.30 | P(Dog) = 0.10		
Box	Confidence (C)	Area(inter)	Area(union)
B1	0.80	0.132	0.158
B2	0.20	0.000	0.080

Trong hình, Area(pred) là diện tích bounding box dự đoán, Area(GT) là diện tích ground-truth box, Area(inter) là diện tích phần giao, và Area(union) là diện tích phần hợp giữa hai box. Các đại lượng này được dùng ở Bước (1) để tính IoU:

$$\text{IoU} = \frac{\text{Area(inter)}}{\text{Area(union)}}.$$

Câu hỏi: Hãy chọn **phát biểu đúng nhất**:

A. Responsible predictor ở training là A_2; còn box có *car score* cao nhất trước NMS ở test time là A_1.

B. Responsible predictor ở training là B_1; còn box có *car score* cao nhất trước NMS ở test time là B_1.

C. Responsible predictor ở training là A_2; còn box có *car score* cao nhất trước NMS ở test time là B_1.

D. Responsible predictor ở training là A_2; còn box có *car score* cao nhất trước NMS ở test time là A_2.

Đáp án: C

Bước 1: Tính IoU với ground-truth
Từ các giá trị Area(inter) và Area(union) trong hình, ta có

$$\text{IoU} = \frac{\text{Area(inter)}}{\text{Area(union)}}.$$

Kết quả:

Box	IoU với GT	Ghi chú
A_1	≈ 0.573	thuộc Cell A
A_2	≈ 0.768	thuộc Cell A
B_1	≈ 0.835	thuộc Cell B
B_2	0	thuộc Cell B

Bước 2: Chọn responsible predictor ở training
Vì **tâm ground-truth nằm trong Cell A**, ở training ta **chỉ xét các box trong Cell A**, tức là A_1 và A_2.
Do

$$\text{IoU}(A_2) > \text{IoU}(A_1),$$

nên $\mathbf{A_2}$ là *responsible predictor*.

Bước 3: Tính car score ở test time
Ở test time,

$$\text{car score} = P(\text{Car} \mid \text{Object}) \times \text{Confidence}_{\text{box}}.$$

Suy ra:

Box	car score
A_1	$0.70 \times 0.55 = 0.385$
A_2	$0.70 \times 0.50 = 0.350$
B_1	$0.60 \times 0.80 = \mathbf{0.480}$
B_2	$0.60 \times 0.20 = 0.120$

Vì B_1 có *car score* lớn nhất, nên box có điểm cao nhất trước NMS ở test time là $\mathbf{B_1}$.

Kết luận:

- Responsible predictor ở training: $\mathbf{A_2}$
- Box có *car score* cao nhất ở test time: $\mathbf{B_1}$

Đáp án: C.

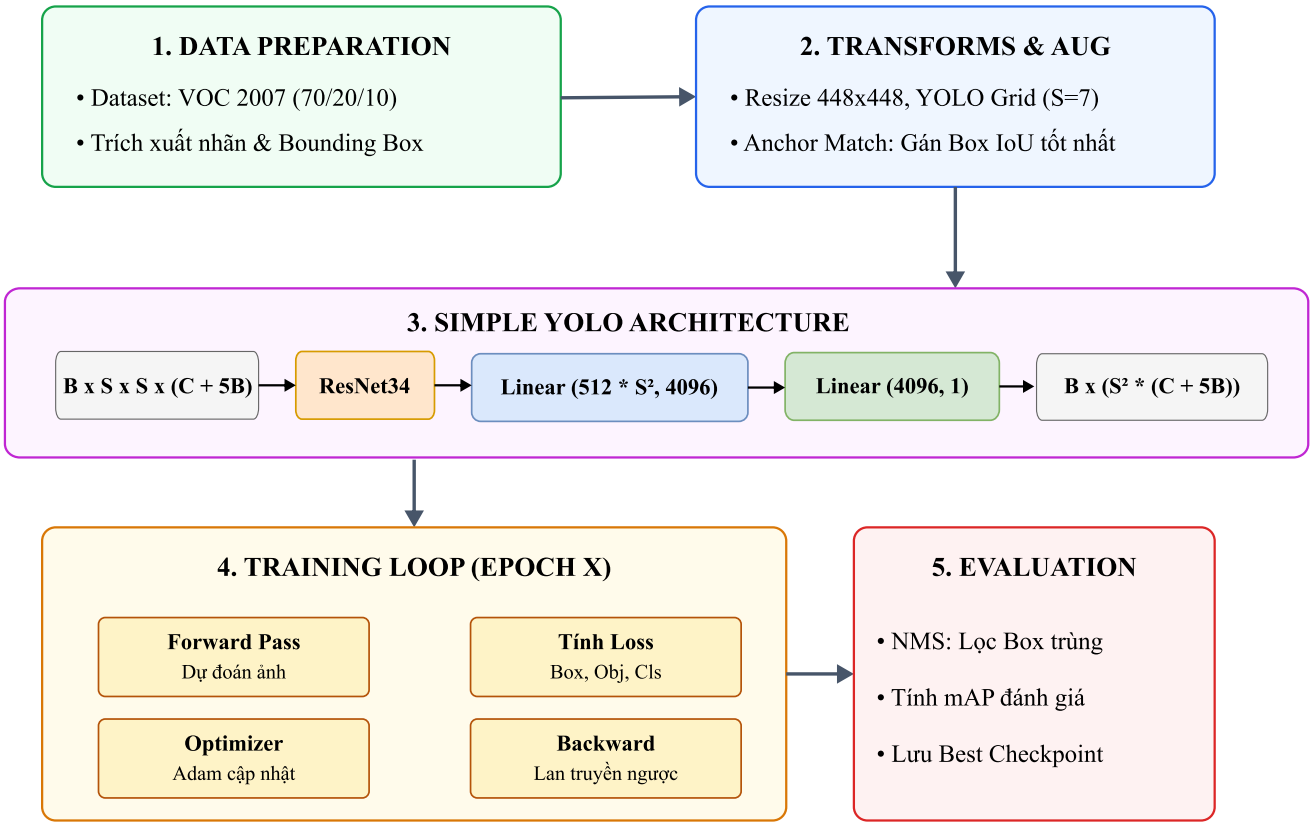
Mô tả bài toán

Nội dung sau đây sẽ được sử dụng cho 3 câu AIO25M09OD03, AIO25M09OD04, AIO25M09OD05

Xét một pipeline YOLO theo hướng **anchor-based**. Khác với YOLOv1 gốc, nơi mỗi cell dự đoán trực tiếp bounding box, thiết kế anchor-based sử dụng các **anchor boxes** có sẵn và mô hình học các offset so với anchor tương ứng. Do đó, pipeline cần thay đổi ở ba bước chính:

- prediction tensor cần được giải mã từ dạng offset–anchor thành các bounding boxes có ý nghĩa hình học;
- các bounding boxes này sau đó được chuyển về dạng danh sách để phục vụ hậu xử lý/evaluation;
- ground-truth annotation cũng phải được mã hóa theo anchor phù hợp để tạo target tensor cho training.

Các câu hỏi sau lần lượt xét các bước quan trọng trong pipeline này.



II.3. AIO25M09OD03 - MEDIUM

Trong pipeline YOLO theo hướng **anchor-based**, tại mỗi grid cell (i, j), mô hình dự đoán:

- một vector lớp dùng chung cho cell:

$$(p_1, p_2, \ldots, p_C),$$

  trong đó p_c là **logit** của lớp c;

- và với mỗi anchor b, một vector:

$$(t_{obj}, t_x, t_y, t_w, t_h),$$

  trong đó:

  - t_{obj}: **objectness logit**,
  - t_x, t_y: offset của tâm box so với cell,
  - t_w, t_h: offset kích thước so với anchor tương ứng.

Do đó, đầu ra của mô hình đang ở **cell-space/anchor-space**, trong khi bước validation cần các bounding box trong **image-space** kèm **class** và **score**. Vì vậy, ta cần giải mã prediction tensor về dạng bounding box chuẩn theo ảnh. Ở câu này, ta xét bước đầu tiên của pipeline: hoàn thiện hàm `convert_cellboxes`.

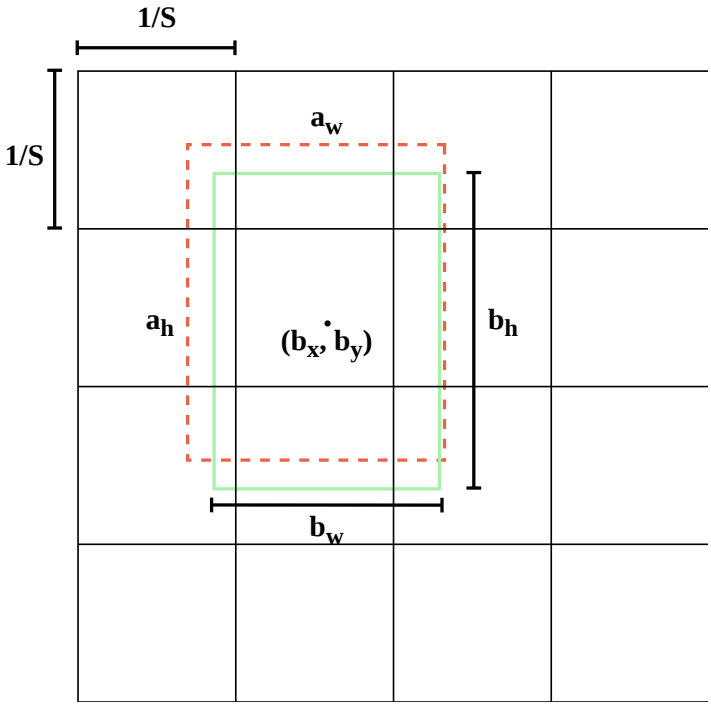
Ta thực hiện việc giải mã theo các bước sau:

1. Chuẩn hóa tọa độ của bounding box về khoảng $[0, 1]$ theo kích thước ảnh:

$$b_x = \frac{\sigma(t_x) + j}{S}, \quad b_y = \frac{\sigma(t_y) + i}{S},$$

$$b_w = \frac{a_w \cdot e^{t_w}}{S}, \quad b_h = \frac{a_h \cdot e^{t_h}}{S}$$

- (a_w, a_h): kích thước của anchor box b
- $\sigma(t_x), \sigma(t_y)$: đưa offset tâm về khoảng $(0, 1)$ trong cell
- e^{t_w}, e^{t_h}: đảm bảo kích thước dương và điều chỉnh theo anchor
- b_x, b_y: tọa độ tâm box đã chuẩn hóa theo ảnh
- b_w, b_h: kích thước box đã chuẩn hóa theo ảnh
- i, j: tọa độ của cell trong grid



Hình 1: Giải mã bounding box từ prediction tensor

2. Vector lớp tại mỗi cell (i, j) được chuẩn hóa bằng softmax:

$$P(c|i,j) = \frac{e^{p_c}}{\sum_{k=1}^{C} e^{p_k}}, \quad \text{class_id}_{i,j} = \arg\max_c P(c|i,j)$$

3. Score của anchor b tại cell (i, j) được tính từ objectness và xác suất lớp lớn nhất:

$$\text{score}_{i,j,b} = \sigma(t_{obj}) \cdot \max_c P(c|i,j)$$

Sau khi giải mã, dự đoán cuối cùng của mỗi anchor có dạng:

$$(\text{class_id}_{i,j},\ \text{score}_{i,j,b},\ b_x,\ b_y,\ b_w,\ b_h)$$

Yêu cầu: Nhiệm vụ của chúng ta là hoàn thiện hàm `convert_cellboxes` như đã mô tả ở trên vào phần `TODO` trong Notebook sau.

Hỏi: giá trị của `sample output[0,0,0]` sau khi chạy testcase dưới đây là gì?

```python
# Test convert_cellboxes
BATCH_SIZE, S, C, B = 2, 7, 20, 2
anchors = torch.tensor([[1.0, 1.0], [2.0, 1.5]], dtype=torch.float32)
predictions = torch.linspace(
    -1.0, 1.0, steps=BATCH_SIZE * S * S * (C + 5 * B)
).reshape(BATCH_SIZE, S * S * (C + 5 * B))

converted = convert_cellboxes(predictions, S=S, C=C, B=B, anchors=anchors)
print("convert_cellboxes output shape:", converted.shape)
print("sample output[0,0,0]:", converted[0, 0, 0])
```

A. `tensor`$\left(\begin{matrix}[[19.0000,\ 0.0137,\ 0.0388,\ 0.0388,\ 0.0534,\ 0.0534],\\ [19.0000,\ 0.0137,\ 0.0389,\ 0.0389,\ 0.1071,\ 0.0804]]\end{matrix}\right)$.

B. `tensor`$\left(\begin{matrix}[[19.0000,\ 0.0214,\ 0.0412,\ 0.0367,\ 0.0601,\ 0.0503],\\ [19.0000,\ 0.0189,\ 0.0405,\ 0.0374,\ 0.0986,\ 0.0721]]\end{matrix}\right)$.

C. `tensor`$\left(\begin{matrix}[[19.0000,\ 0.0095,\ 0.0341,\ 0.0402,\ 0.0478,\ 0.0589],\\ [19.0000,\ 0.0152,\ 0.0427,\ 0.0413,\ 0.1124,\ 0.0856]]\end{matrix}\right)$.

D. `tensor`$\left(\begin{matrix}[[19.0000,\ 0.0176,\ 0.0372,\ 0.0394,\ 0.0558,\ 0.0497],\\ [19.0000,\ 0.0121,\ 0.0369,\ 0.0375,\ 0.1035,\ 0.0779]]\end{matrix}\right)$.

Đáp án: A

Source Code đáp án: Notebook

II.4. AIO25M09OD04 - EASY

Sau khi hoàn thiện hàm `convert_cellboxes` ở câu AIO25M09OD03, mỗi anchor tại mỗi grid cell đã được giải mã về dạng

$$(\text{class_id}_{i,j},\ \text{score}_{i,j,b},\ b_x,\ b_y,\ b_w,\ b_h).$$

Tuy nhiên, không phải mọi bounding box dự đoán đều đủ tin cậy để đưa vào bước evaluation hay hậu xử lý. Vì vậy, ta tiếp tục thực hiện bước **lọc theo ngưỡng score** để chỉ giữ lại các box có độ tin cậy đủ cao:

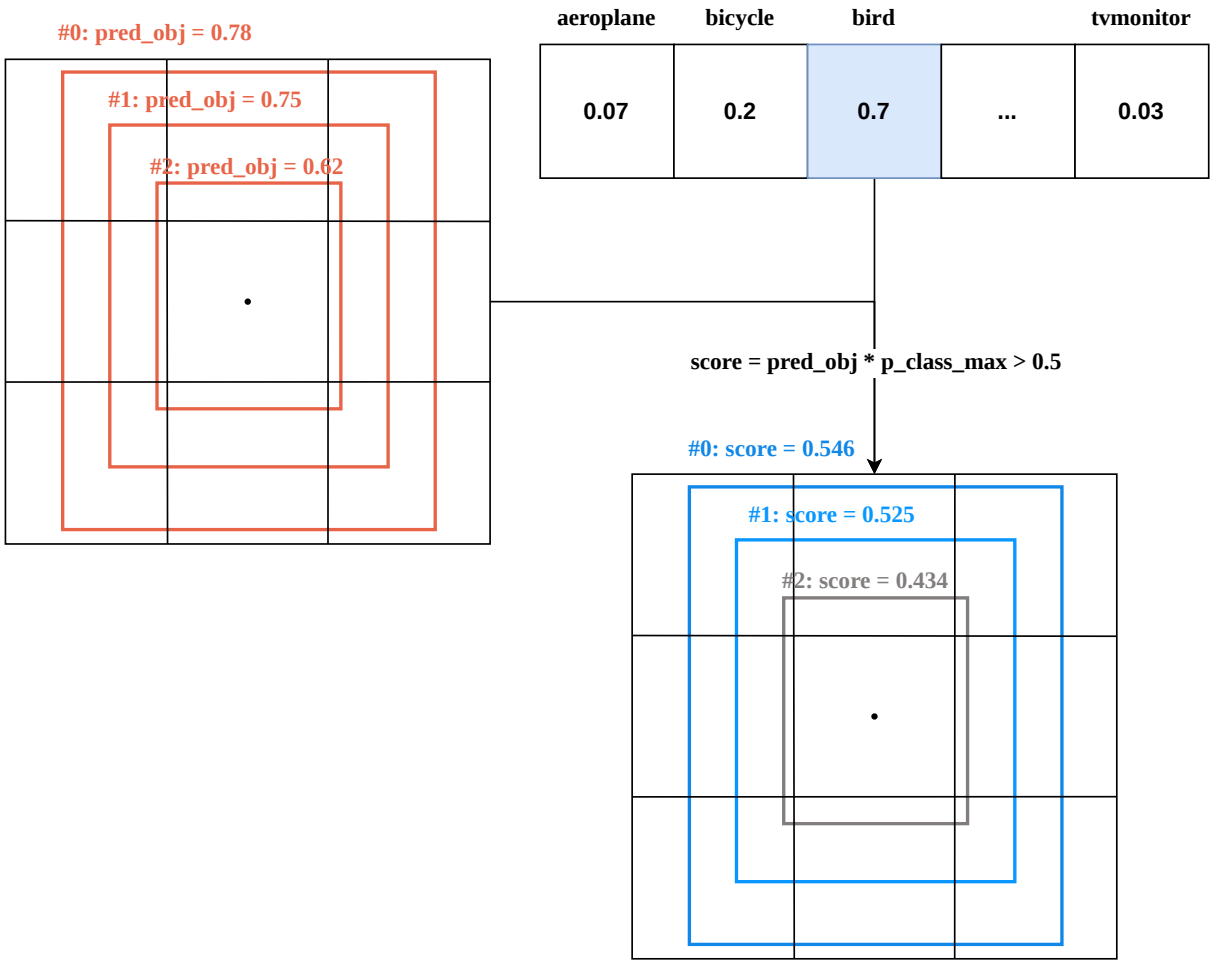
$$\text{score}_{i,j,b} > \tau$$

trong đó τ là ngưỡng lọc (ví dụ $\tau = 0.5$).

Những bounding box thỏa điều kiện trên sẽ được đưa vào danh sách các dự đoán hợp lệ:

$$\mathcal{B} = \left\{ \left(\text{class_id}_{i,j},\ \text{score}_{i,j,b},\ b_x,\ b_y,\ b_w,\ b_h\right) \;\middle|\; \text{score}_{i,j,b} > \tau \right\}$$

Tập $\mathcal{B}$ chứa toàn bộ các bounding box dự đoán có độ tin cậy đủ cao trên ảnh và sẽ được sử dụng cho các bước tiếp theo như *validation* hoặc *Non-Maximum Suppression (NMS)* để loại bỏ các box trùng lặp và giữ lại những dự đoán tốt nhất.



Hình 2: Quá trình lọc bounding box theo ngưỡng score τ

Yêu cầu: Nhiệm vụ của chúng ta là hoàn thiện hàm `cellboxes_to_boxes` như đã mô tả ở trên vào phần `TODO` trong Notebook sau. Sau khi chạy testcase dưới đây, giá trị của `sample box from image 0` là gì?

```
# Test cellboxes_to_boxes
BATCH_SIZE, S, C, B = 2, 7, 20, 2
anchors = torch.tensor([[1.0, 1.0], [2.0, 1.5]], dtype=torch.float32)
predictions = torch.linspace(
    -1.0, 1.0, steps=BATCH_SIZE * S * S * (C + 5 * B)
).reshape(BATCH_SIZE, S * S * (C + 5 * B))

boxes = cellboxes_to_boxes(predictions, S=S, C=C, B=B, anchors=anchors, threshold=0.01)
print("number of images:", len(boxes))
print("number of boxes in image 0:", len(boxes[0]))
print(
    "sample box from image 0:",
    [round(x, 4) if isinstance(x, float) else x for x in boxes[0][0]]
    if len(boxes[0]) > 0 else "no boxes kept"
)
```

A. [19, 0.0182, 0.0415, 0.0369, 0.0591, 0.0507].

B. [19, 0.0137, 0.0388, 0.0388, 0.0534, 0.0534].

C. [19, 0.0104, 0.0352, 0.0401, 0.0486, 0.0583].

D. [19, 0.0168, 0.0374, 0.0396, 0.0562, 0.0479].

Đáp án: B

Source Code đáp án: Notebook

II.5. AIO25M09OD05 - HARD

Khác với hai câu trước (AIO25M09OD03 và AIO25M09OD04) tập trung vào prediction, ở câu này ta xét phía **dataset/target** trong pipeline YOLO theo hướng **anchor-based**. Khi mô hình dự đoán theo các anchor boxes và offset tương ứng, ground-truth annotation cũng phải được mã hóa lại thành **target tensor** để phục vụ training.

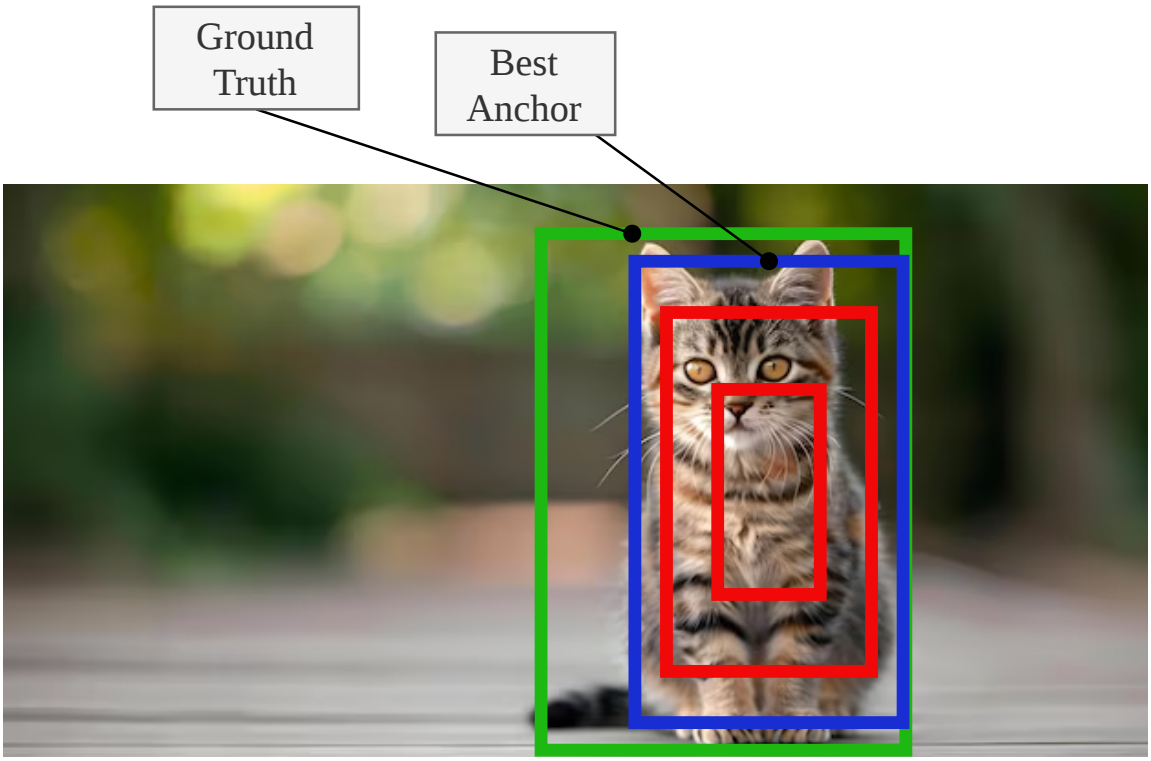
Cụ thể, mỗi sample trong dataset trả về một cặp:

$$(\text{image},\ \text{label_matrix}), \quad \text{image} \in \mathbb{R}^{3\times H\times W}, \quad \text{label_matrix} \in \mathbb{R}^{S\times S\times (C+5B)}.$$

Tại mỗi cell (i, j), label_matrix được tổ chức như sau:

$$[\text{class one-hot } (C) \mid \underbrace{(o, t_x, t_y, t_w, t_h)}_{\text{anchor 1}} \mid \ldots \mid \underbrace{(o, t_x, t_y, t_w, t_h)}_{\text{anchor B}}].$$

Với mỗi ground-truth object, ta cần xác định cell chứa tâm, chọn anchor có IoU cao nhất, rồi gán class label, objectness và các tham số box vào đúng vị trí trong target tensor.



Chỉ anchor có IoU cao nhất với Ground Truth được gán objectness = 1

Yêu cầu: Nhiệm vụ của chúng ta là hoàn thiện hàm `__getitem__` của `CustomVOCDataset` tại phần `TODO` trong ô `Custom Dataset Class` của Notebook theo nội dung mô tả ở trên.

Sau đó, chạy notebook với cấu hình mặc định trong **2 epoch**. Giá trị `best_mAP_test` thu được nằm trong khoảng nào sau đây?

Lưu ý: Trước khi chạy câu này, hãy sao chép phần cài đặt hoàn chỉnh của hai hàm `convert_cellboxes` và `cellboxes_to_boxes` từ hai câu trước (AIO25M09OD03 và AIO25M09OD04) vào ô `Put code from previous questions` trong Colab.

A. (0.0030, 0.0040].

B. (0.0040, 0.0050].

C. (0.0050, 0.0060].

D. (0.0060, 0.0070].

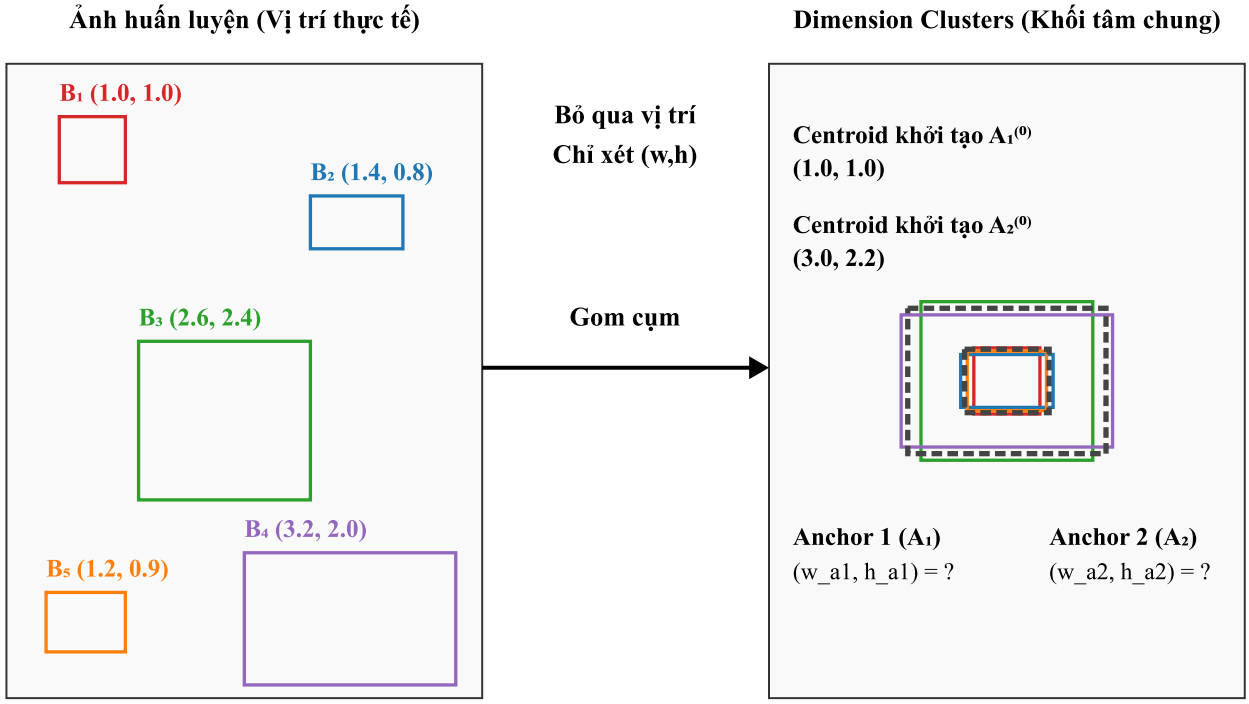
Đáp án: B

Source Code đáp án (bao gồm Training): Notebook

II.6. AIO25M09OD06 - MEDIUM

Trong YOLOv2, trước khi huấn luyện mô hình phát hiện vật thể, ta chạy **k-means clustering** trên các ground-truth bounding boxes để tìm ra **prior anchors**. Các prior anchors này đóng vai trò là **các box tham chiếu ban đầu** trong giai đoạn training, để mô hình học **offsets** từ prior box tới bounding box thật của object.

Ở bước này, YOLOv2 **bỏ qua vị trí tâm** (x, y) và chỉ sử dụng **kích thước** (w, h) của từng box để phân cụm. Vì vậy, **vị trí các box trong hình chỉ mang tính minh họa, không được dùng trong tính toán**. Hình dưới minh họa 5 ground-truth bounding boxes với các kích thước (w, h) tương ứng, cùng với **2 centroid khởi tạo ban đầu**.



Ta chạy **k-means với** $k = 2$. Ở mỗi vòng lặp:

- Gán mỗi ground-truth box vào centroid có khoảng cách nhỏ hơn.
- Cập nhật mỗi centroid bằng **trung bình cộng** các cặp (w, h) thuộc cụm đó.
- Lặp lại cho đến khi các cụm không đổi.

Khoảng cách giữa một box (w, h) và một anchor (a_w, a_h) được định nghĩa bởi:

$$d\Big((w,h),(a_w,a_h)\Big) = 1 - \text{IoU}\Big((w,h),(a_w,a_h)\Big)$$

với

$$\text{IoU}\Big((w,h),(a_w,a_h)\Big) = \frac{\min(w,a_w)\min(h,a_h)}{wh + a_w a_h - \min(w,a_w)\min(h,a_h)}$$

Câu hỏi: Sau khi chạy k-means đến khi hội tụ, hai prior anchors thu được là:

A. $(1.1, 0.9)$ và $(3.0, 2.2)$.

B. $(1.2, 0.9)$ và $(2.9, 2.2)$.

C. $(1.2, 1.0)$ và $(2.8, 2.4)$.

D. $(1.4, 0.8)$ và $(3.2, 2.0)$.

Đáp án: B

Ta chạy **k-means với** $k = 2$ trên **kích thước** (w, h) của 5 ground-truth boxes trong hình, **không dùng vị trí tâm** (x, y). Hai centroid khởi tạo là

$$A_1^{(0)} = (1.0, 1.0), \qquad A_2^{(0)} = (3.0, 2.2).$$

Bước 1: Gán cụm bằng khoảng cách $1 - \text{IoU}$
Ví dụ, với box $(1.4, 0.8)$ và centroid $(1.0, 1.0)$:

$$\text{IoU} = \frac{\min(1.4, 1.0)\min(0.8, 1.0)}{1.4 \cdot 0.8 + 1.0 \cdot 1.0 - \min(1.4, 1.0)\min(0.8, 1.0)} = \frac{1.0 \cdot 0.8}{1.12 + 1.0 - 0.8} = \frac{0.8}{1.32} \approx 0.606,$$

nên

$$d = 1 - \text{IoU} \approx 0.394.$$

Tính tương tự cho các box còn lại, ta thu được kết quả gán cụm đầu tiên:

$$\mathcal{C}_1 = \{(1.0, 1.0),\ (1.4, 0.8),\ (1.2, 0.9)\},$$

$$\mathcal{C}_2 = \{(2.6, 2.4),\ (3.2, 2.0)\}.$$

Bước 2: Cập nhật centroid bằng trung bình cộng

$$A_1^{(1)} = \left(\frac{1.0 + 1.4 + 1.2}{3}, \frac{1.0 + 0.8 + 0.9}{3}\right) = (1.2, 0.9),$$

$$A_2^{(1)} = \left(\frac{2.6 + 3.2}{2}, \frac{2.4 + 2.0}{2}\right) = (2.9, 2.2).$$

Bước 3: Kiểm tra hội tụ
Dùng hai centroid mới để gán cụm lại, ta thu được đúng các cụm cũ, nên thuật toán **hội tụ**.

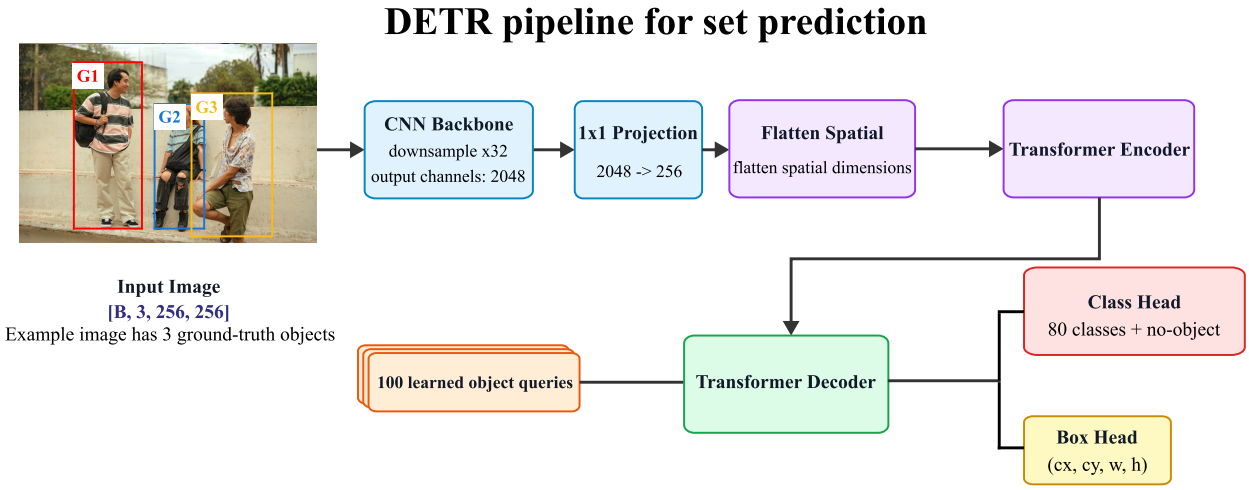
Kết luận: Hai prior anchors cuối cùng là

$$(1.2, 0.9) \quad \text{và} \quad (2.9, 2.2).$$

Link đáp án chạy từng bước Notebook

II.7. AIO25M09OD07 - EASY

Trong DETR, ảnh đầu vào được đưa qua backbone CNN để tạo feature map, sau đó feature map được project về hidden dimension, flatten thành chuỗi token không gian và đưa vào transformer encoder. Ở phía decoder, mô hình sử dụng **learned object queries** để tạo ra một **fixed-size set** các prediction outputs.



Dựa trên pipeline trong hình, biết thêm rằng:

- Batch size $B = 2$.
- Sau khi flatten, Encoder giữ nguyên số token và hidden dimension.
- Mỗi object query tạo ra đúng 1 decoder output.
- Với **một ảnh** có đúng **3 ground-truth objects**, sau Hungarian matching thì các prediction slots còn lại được gán vào lớp $\varnothing$.

Câu hỏi: shape của chuỗi token không gian đưa vào Encoder, shape đầu ra của Class Head, và số lượng prediction slots được gán vào lớp $\varnothing$ lần lượt là bao nhiêu?

A. $[2, 256, 8, 8]$, $[2, 100, 80]$, 97.

B. $[2, 64, 256]$, $[2, 100, 81]$, 61.

C. $[2, 64, 256]$, $[2, 100, 81]$, 97.

D. $[2, 64, 2048]$, $[2, 81, 100]$, 96.

Đáp án: C

Đầu tiên, ta xác định shape của chuỗi token đưa vào Encoder:

- Backbone giảm kích thước không gian đi 32 lần, nên chiều dài và chiều rộng của feature map là $256 \div 32 = 8$.
- Lớp 1×1 projection giảm số kênh từ $2048 \rightarrow 256$.
- Quá trình flatten (trải phẳng) theo không gian 8×8 sẽ tạo ra chuỗi gồm $8 \times 8 = 64$ tokens. Vậy shape đưa vào Encoder là:

$$[2, 64, 256].$$

Tiếp theo, ta xác định shape đầu ra của Class Head:

- Decoder sử dụng $N = 100$ learned object queries, sinh ra 100 prediction outputs.
- Class head dự đoán phân phối xác suất cho C lớp đối tượng cộng thêm 1 lớp $\varnothing$, tức là $80 + 1 = 81$ logits cho mỗi query. Vậy shape là:

$$[2, 100, 81].$$

Cuối cùng, ta tính số lượng slots được gán vào lớp $\varnothing$:

- Trong một ảnh có đúng 3 ground-truth objects, thuật toán Hungarian matching sẽ ghép 3 prediction slots với 3 đối tượng thật này.
- Các slots còn lại (trong tổng số 100 slots đầu ra của Decoder) sẽ bị gán vào lớp $\varnothing$ (no-object):

$$100 - 3 = 97.$$

Lưu ý: Không được lấy số token của Encoder (64) trừ đi 3, vì Hungarian matching diễn ra ở đầu ra của Decoder (nơi có 100 queries). Vậy đáp án đúng là **(c)**.

Mô tả bài toán

Hãy dựa vào **hàm loss** và **cách mô hình YOLOv1 mã hoá đầu ra (output tensor)** được minh hoạ trong hình dưới đây để trả lời các câu hỏi từ **AIO25M09OD08** đến **AIO25M09OD11**.

$$L_{loc} = \sum_{j=0}^{B} 1_j^{obj} * [(x - \hat{x})^2 + (y - \hat{y})^2]]$$
$$+ \sum_{j=0}^{B} 1_j^{obj} * [(\sqrt{w} - \sqrt{\hat{w}})^2 + (\sqrt{h} - \sqrt{\hat{h}})^2]$$

$$L_{obj} = \sum_{j=0}^{B} 1_j^{obj} * (C - \hat{C})^2 , with\ C = 1$$

$$L_{no-obj} = \sum_{j=0}^{B} 1_j^{no-obj} * (C - \hat{C})^2 , with\ C = 0$$

$$L_{class} = 1^{obj} * \sum_{c=0}^{\#classes} (p(c) - \hat{p}(c))^2$$

$$L = \lambda_{loc} * L_{loc} + \lambda_{obj} * L_{obj} + \lambda_{no-obj} * L_{no-obj} + \lambda_{class} * L_{class}$$

1_j^{obj}: Nếu cùng 1 cell chọn box có IoU (so với targe bbox) cao nhất và cell có chứa object (dựa vào target)
1^{obj}: Cell có chứa object (dựa vào target)

Trong YOLOv1, mỗi bounding box thường gồm 5 thành phần $(x, y, w, h, \text{confidence})$. Tuy nhiên, trong chuỗi câu hỏi này, **tuỳ theo giả thiết cụ thể của từng bài**, ta xét trường hợp mô hình có thể được **rút gọn đầu ra**, tức là chỉ giữ lại những thành phần thực sự cần thiết để mô tả object trong bối cảnh đã cho.

Gọi k là **số thành phần thực sự cần dự đoán cho mỗi bounding box** trong từng câu. Khi đó, output shape của mô hình được xác định theo:

$$\Big(N,\ S \times S \times (B \times k + C)\Big)$$

Trong đó:

- N là số lượng ảnh trong một batch,
- $S \times S$ là số ô lưới,
- B là số bounding box dự đoán tại mỗi ô,
- C là số lớp object,
- k phụ thuộc vào giả thiết cụ thể của từng câu.

II.8. AIO25M09OD08 - EASY

Hãy xác định **output shape** của mô hình là bao nhiêu?

Giả thiết bài toán:

- Dataset bao gồm **20 loại object** khác nhau.
- Mỗi ảnh có thể chứa **một object duy nhất** hoặc **không chứa object nào**.
- Ảnh được chia thành **một ô duy nhất** ($S = 1$).
- Mô hình dự đoán **3 bounding box** cho mỗi ảnh ($B = 3$).
- Các object có **kích thước khác nhau**, không đồng nhất.

A. (N, 25).

B. (N, 15).

C. (N, 35).

D. (N, 45).

Đáp án: C

Ta có:

$$S = 1, \quad B = 3, \quad C = 20$$

Theo mô tả bài toán, k là **số thành phần thực sự cần dự đoán cho mỗi bounding box**.
Trong câu này, object có thể xuất hiện hoặc không xuất hiện và các object có kích thước khác nhau, nên mỗi bounding box vẫn cần giữ đủ:

$$(x, y, w, h, \text{confidence})$$

tức là:

$$k = 5$$

Vậy output shape được tính theo:

$$(N,\ S \times S \times (B \times k + C))$$

Thay số vào:

$$(N,\ 1 \times 1 \times (3 \times 5 + 20))$$

$$(N,\ 1 \times (15 + 20)) = (N, 35)$$

Kết luận: **output shape của mô hình là** $(N, 35)$.

II.9. AIO25M09OD09 - EASY

Hãy xác định **output shape** của mô hình là bao nhiêu?

Giả thiết bài toán:

- Dataset gồm **20 loại object**.
- **Mỗi ảnh luôn chứa đúng 1 object**.
- Ảnh được chia thành **1 ô duy nhất** ($S = 1$).
- Mô hình dự đoán **3 bounding box** ($B = 3$).
- Các object có **kích thước khác nhau**.

A. (N, 30).

B. (N, 32).

C. (N, 35).

D. (N, 40).

Đáp án: B

Ta có:

$$S = 1, \quad B = 3, \quad C = 20$$

Theo mô tả bài toán, k là **số thành phần thực sự cần dự đoán cho mỗi bounding box**.
Trong câu này, mỗi ảnh **luôn chứa đúng 1 object**. Theo quy ước rút gọn của bài toán, ta chỉ cần mỗi bounding box dự đoán:

$$(x, y, w, h)$$

nên:

$$k = 4$$

Vậy output shape được tính theo:

$$(N,\ S \times S \times (B \times k + C))$$

Thay số vào:

$$(N,\ 1 \times 1 \times (3 \times 4 + 20))$$

$$(N,\ 1 \times (12 + 20)) = (N, 32)$$

Kết luận: **output shape của mô hình là** $(N, 32)$.

II.10. AIO25M09OD10 - EASY

Hãy xác định **output shape** của mô hình là bao nhiêu?

Giả thiết bài toán:

- Dataset bao gồm **20 loại object** khác nhau.
- Mỗi ảnh có thể chứa **một object duy nhất** hoặc **không chứa object nào**.
- Ảnh được chia thành **một ô duy nhất** ($S = 1$).
- Mô hình dự đoán **3 bounding box** cho mỗi ảnh ($B = 3$).
- Tất cả object có **cùng size 32×32**.

A. (N, 29).

B. (N, 32).

C. (N, 35).

D. (N, 40).

Đáp án: A

Ta có:

$$S = 1, \quad B = 3, \quad C = 20$$

Theo mô tả bài toán, k là **số thành phần thực sự cần dự đoán cho mỗi bounding box**.
Trong câu này, mọi object đều có cùng kích thước 32×32, nên theo quy ước rút gọn của bài toán, mô hình không cần dự đoán riêng w, h cho từng bounding box. Khi đó mỗi bounding box chỉ cần:

$$(x, y, \text{confidence})$$

nên:

$$k = 3$$

Vậy output shape được tính theo:

$$(N,\ S \times S \times (B \times k + C))$$

Thay số vào:

$$(N,\ 1 \times 1 \times (3 \times 3 + 20))$$

$$(N,\ 1 \times (9 + 20)) = (N, 29)$$

Kết luận: **output shape của mô hình là** $(N, 29)$.

II.11. AIO25M09OD11 - EASY

Hãy xác định **output shape** của mô hình là bao nhiêu?

Giả thiết bài toán:

- Dataset bao gồm **20 loại object** khác nhau.
- Mỗi ảnh **luôn có một object**.
- Ảnh được chia thành **một ô duy nhất** ($S = 1$).
- Mô hình dự đoán **3 bounding box** cho mỗi ảnh ($B = 3$).
- Các object có **cùng size 32×32**.

A. (N, 20).

B. (N, 26).

C. (N, 29).

D. (N, 35).

Đáp án: B

Ta có:

$$S = 1, \quad B = 3, \quad C = 20$$

Theo mô tả bài toán, k là **số thành phần thực sự cần dự đoán cho mỗi bounding box**.
Trong câu này:

- mỗi ảnh luôn có một object;
- tất cả object đều có cùng kích thước 32×32.

Theo quy ước rút gọn của bài toán, khi đó mỗi bounding box chỉ cần dự đoán vị trí tâm:

$$(x, y)$$

nên:

$$k = 2$$

Vậy output shape được tính theo:

$$(N,\ S \times S \times (B \times k + C))$$

Thay số vào:

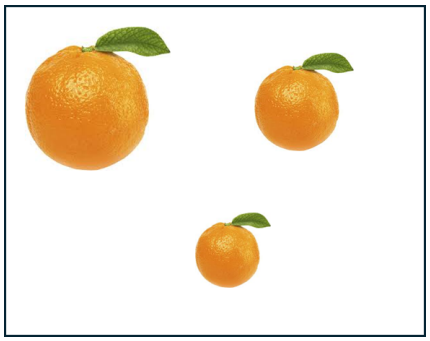
$$(N,\ 1 \times 1 \times (3 \times 2 + 20))$$

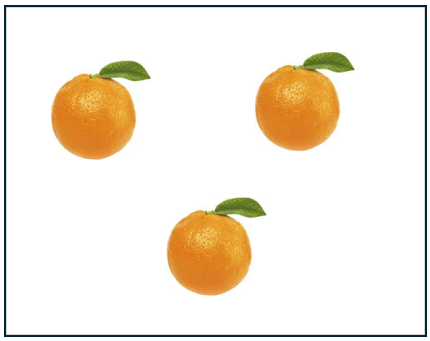
$$(N,\ 1 \times (6 + 20)) = (N, 26)$$

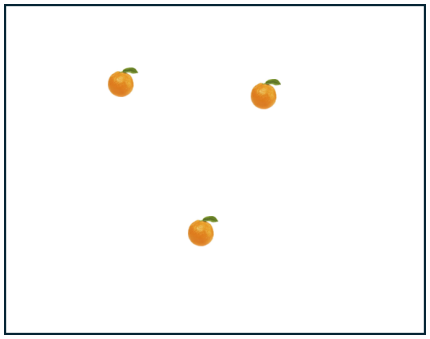
Kết luận: **output shape của mô hình là** $(N, 26)$.

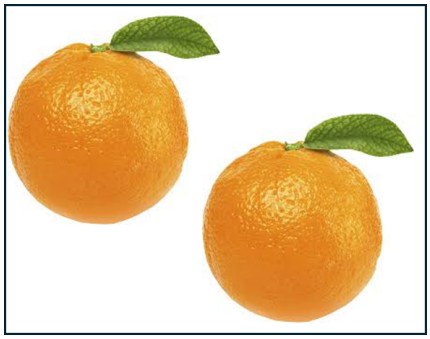
II.12. AIO25M09OD12 - EASY

YOLOv3 và YOLOv5 dùng thêm kỹ thuật *multi-scale detection* sẽ giúp cải thiện độ chính xác **nhiều nhất** cho ảnh nào sau đây?

A. 

C. 

B. 

D. 

Đáp án: A

Kỹ thuật multi-scale detection trong YOLOv3 và YOLOv5 cho phép mô hình phát hiện các đối tượng ở nhiều kích thước khác nhau, từ nhỏ đến lớn. Điều này rất quan trọng trong các bức ảnh có đối tượng có kích thước khác nhau, như trong trường hợp của bức ảnh A, nơi có quả cam nhỏ, vừa và lớn. Các bức ảnh B, C và D có đối tượng có kích thước tương đối đồng nhất, do đó không cần thiết phải sử dụng multi-scale detection.

III. Object Tracking

III.1. AIO25M09OT01 - EASY.

Tại khung hình t, bộ lọc dự đoán xe ở tọa độ $y_t = 150$. Tuy nhiên, `YOLO26` phát hiện xe ở vị trí $y_p = 180$. Nếu Kalman Gain $K_x = 0.3$, tọa độ được cập nhật cuối cùng là gì, biết rằng

$$y_{t+1} = y_t + K_x(y_p - y_t)$$

A. 159.

B. 165.

C. 171.

D. 180.

Đáp án: A

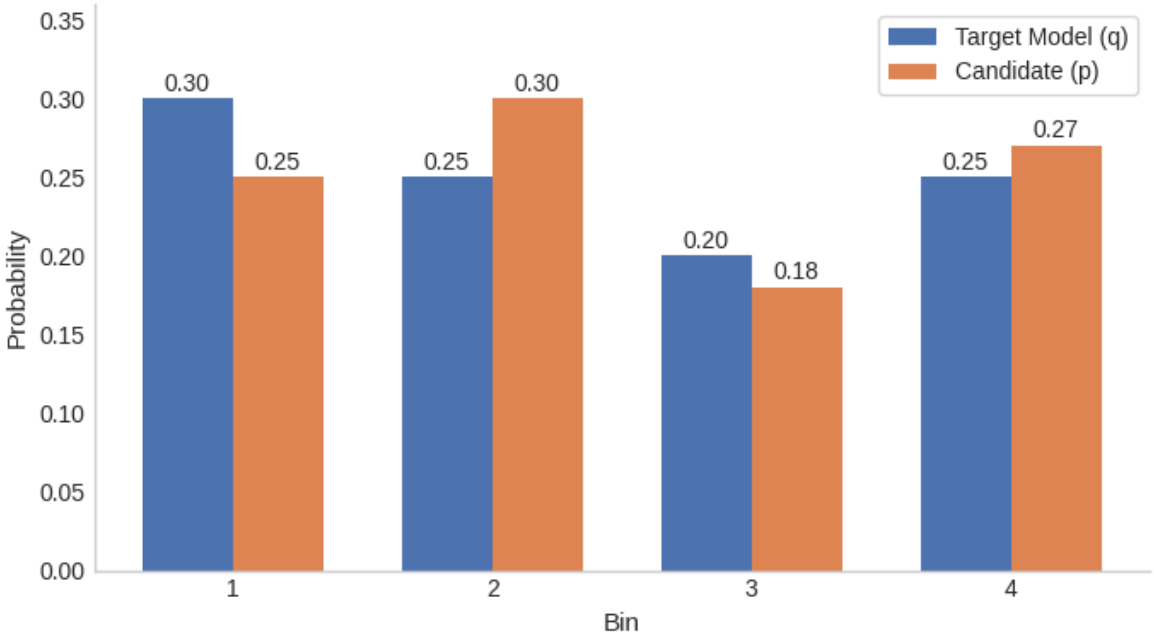
Giải thích: Dựa vào công thức đã cho, ta có:

$$\begin{aligned} y_{t+1} &= y_t + K_x(y_p - y_t) \\ &= 150 + 0.3(180 - 150) \\ &= 159 \end{aligned}$$

Như vậy, vị trí sau khi được cập nhật là $y_{t+1} = 159$. Do đó, đáp án đúng là A.

III.2. AIO25M09OT02 - EASY.

Hệ thống đang theo dõi một vật thể trong video. Bên dưới là histogram của mô hình mục tiêu (Target Model q) và vùng ảnh ứng viên hiện tại (Candidate p). Histogram được chia thành 4 bin chính như sau:



Dựa vào công thức tính hệ số Bhattacharyya:

$$\mathrm{BC}(p,q) = \sum_{i=1}^{m} \sqrt{p_i q_i}$$

trong đó m là số *bin* của histogram. Hãy tính hệ số tương đồng giữa hai phân phối trên. Ngoài ra, giả sử ngưỡng tương đồng tối thiểu để tiếp tục theo dõi trong thuật toán *Mean Shift Object Tracking* là 0.95. Thuật toán có tiếp tục theo dõi vật thể hiện tại hay không?

A. $\mathrm{BC}(p,q) \approx 1.00$, không tiếp tục theo dõi.

B. $\mathrm{BC}(p,q) \approx 0.88$, tiếp tục theo dõi.

C. $\mathrm{BC}(p,q) \approx 0.96$, không tiếp tục theo dõi.

D. $\mathrm{BC}(p,q) \approx 1.00$, tiếp tục theo dõi.

Đáp án: D

Giải thích: Từ histogram trong Hình ở đề bài, ta có:

- $p = [0.30, 0.25, 0.20, 0.25]$

- $q = [0.25, 0.30, 0.18, 0.27]$

Áp dụng công thức tính hệ số Bhattacharyya, ta có:

$$\begin{aligned} \text{BC}(p, q) &= \sqrt{0.30 \times 0.25} + \sqrt{0.25 \times 0.30} + \sqrt{0.20 \times 0.18} + \sqrt{0.25 \times 0.27} \\ &\approx 0.274 + 0.274 + 0.190 + 0.260 \\ &\approx 0.998 \approx 1.00 \end{aligned}$$

Vì $\text{BC}(p, q) \approx 0.998 > 0.95$, nên thuật toán *Mean Shift Object Tracking* vẫn tiếp tục theo dõi vật thể. Do đó, đáp án đúng là D.

III.3. AIO25M09OT03 - EASY.

Dựa vào cấu hình của ByteTracker trong đoạn code dưới đây, giả sử video có tốc độ 24 FPS. Hỏi thời gian tối đa mà hệ thống vẫn có thể giữ ID cũ cho một đối tượng khi đối tượng đó bị che khuất (occlusion) là bao lâu?

Ngoài ra, giả sử một chiếc xe bị vật cản che khuất trong vòng 1.5 giây. Hệ thống có còn giữ nguyên ID cũ của chiếc xe khi nó xuất hiện trở lại hay không?

```python
import supervision as sv

byte_tracker = sv.ByteTrack(
    track_activation_threshold=0.25,
    lost_track_buffer=48,
    minimum_matching_threshold=0.8,
    frame_rate=24,
    minimum_consecutive_frames=3
)
```

A. 1 giây, Có.

B. 2 giây, Có.

C. 3 giây, Không.

D. 48 giây, Không.

Đáp án: B

Giải thích: Trong `ByteTrack` của thư viện `supervision`, tham số `lost_track_buffer` quy định số lượng frame tối đa mà một đối tượng có thể không được phát hiện trước khi tracker xoá track đó.

Do đó, thời gian tối đa mà một đối tượng có thể bị che khuất mà vẫn giữ được ID cũ là

$$\text{Max_occlusion_time} = \frac{\text{lost_track_buffer}}{\text{frame_rate}} = \frac{48}{24} = 2 \text{ giây}$$

Vì chiếc xe chỉ bị che khuất trong 1.5 giây (< 2 giây), nên hệ thống vẫn giữ nguyên ID cũ khi chiếc xe xuất hiện trở lại. Do đó đáp án đúng là B.

III.4. AIO25M09OT04 - MEDIUM.

Dựa vào cấu hình của DeepSORT trong đoạn code dưới đây:

```python
from deep_sort_realtime.deepsort_tracker import DeepSort

tracker = DeepSort(
    max_age=30,
    n_init=3,
    max_cosine_distance=0.2
)
```

Tại thời điểm t, hệ thống phát hiện một đối tượng và trích xuất được vector đặc trưng:

$$x = [1, 2, 0]$$

Trước đó, hệ thống đã lưu feature embedding của một track đang tồn tại:

$$y = [1, 1, 0]$$

Biết rằng:

$$\text{cosine similarity} = \frac{x \cdot y}{\|x\|\|y\|}, \quad \text{cosine distance} = 1 - \text{cosine similarity}$$

Hỏi cosine distance giữa x và y là bao nhiêu? Và DeepSORT có ghép detection này với track cũ y hay không?

A. 0.05, Có.

B. 0.11, Có.

C. 0.25, Không.

D. 0.50, Không.

Đáp án: A

Giải thích: Trước hết, ta có:

- $\|x\| = \sqrt{1^2 + 2^2 + 0^2} = \sqrt{5}$
- $\|y\| = \sqrt{1^2 + 1^2 + 0^2} = \sqrt{2}$

Tích vô hướng giữa x và y là

$$x \cdot y = 1 \times 1 + 2 \times 1 + 0 \times 0 = 3$$

Từ đó, dựa vào công thức tính, ta có Cosine Similarity giữa 2 embedding x và y là:

$$\text{cosine similarity} = \frac{x \cdot y}{\|x\|\|y\|} = \frac{3}{\sqrt{5}\sqrt{2}} = \frac{3}{\sqrt{10}}$$

Như vậy khoảng cách Cosine giữa x và y là

$$\text{cosine distance} = 1 - \text{cosine similarity} = 1 - \frac{3}{\sqrt{10}} \approx 0.05$$

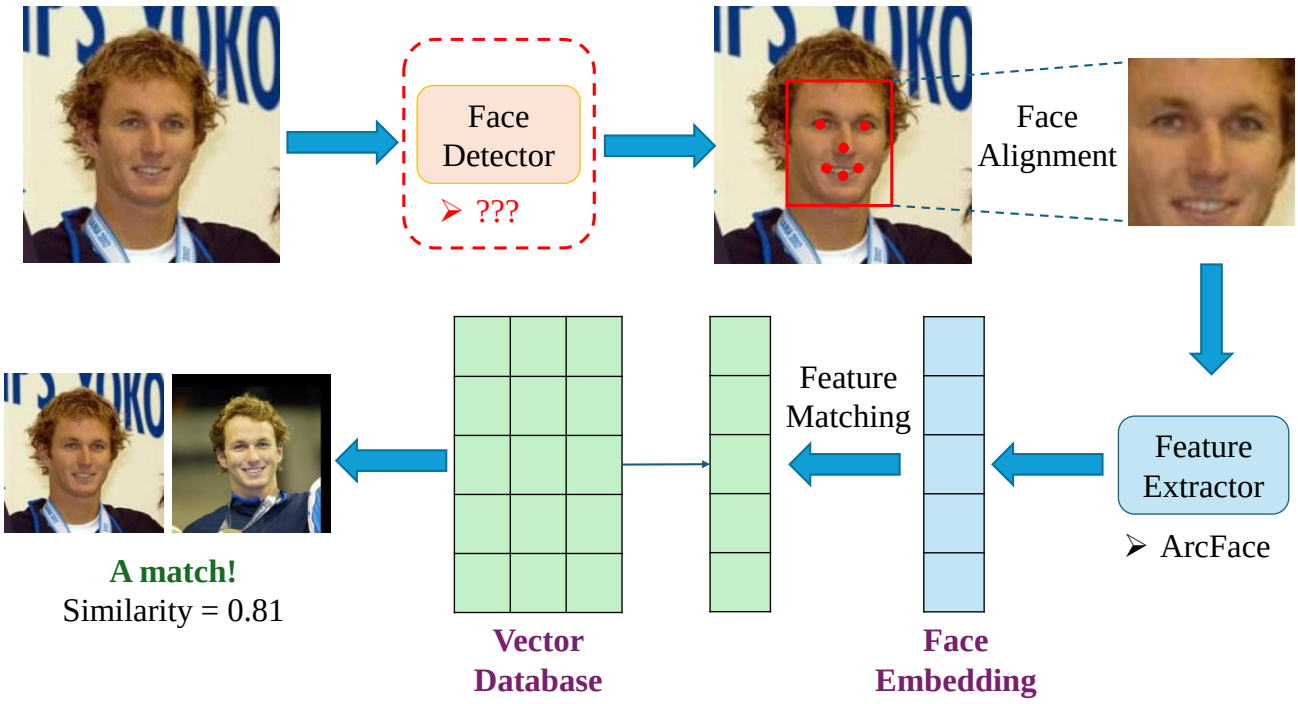
Trong DeepSORT của thư viện `deep_sort_realtime`, tham số `max_cosine_distance` quy định ngưỡng tối đa của cosine distance để hai embedding được xem là cùng một đối tượng.

Vì cosine distance giữa x và y là 0.05, nhỏ hơn 0.2, nên detection với embedding x vẫn được ghép với track cũ có embedding y. Như vậy, đáp án đúng là A.

IV. Face Recognition

IV.1. AIO25M09FACE01 - EASY

Bạn mong muốn xây dựng một hệ thống Face Recognition với pipeline bao gồm: Face Detection, Face Alignment, Feature Extraction, và Feature Matching như hình minh họa bên dưới.



Hiện tại, bạn đang tìm kiếm mô hình phù hợp cho bước Face Detector. Bạn mong muốn sử dụng các mô hình trên repository của `insightface` và dựa vào các kết quả thực nghiệm của họ để chọn mô hình (tại Github). Tiêu chí của bạn là $Flops \leq 10G$ và trong số những mô hình thỏa điều kiện, chọn mô hình có độ chính xác tốt nhất. Trong các mô hình dưới đây, đâu là mô hình phù hợp nhất?

A. SCRFD-34GF.

B. SCRFD-10GF.

C. SCRFD-2.5GF.

D. SCRFD-0.5GF.

Đáp án: B

Giải thích: Tiêu chí lựa chọn là $FLOPs \leq 10G$ và trong các mô hình thỏa điều kiện, chọn mô hình có độ chính xác cao nhất.

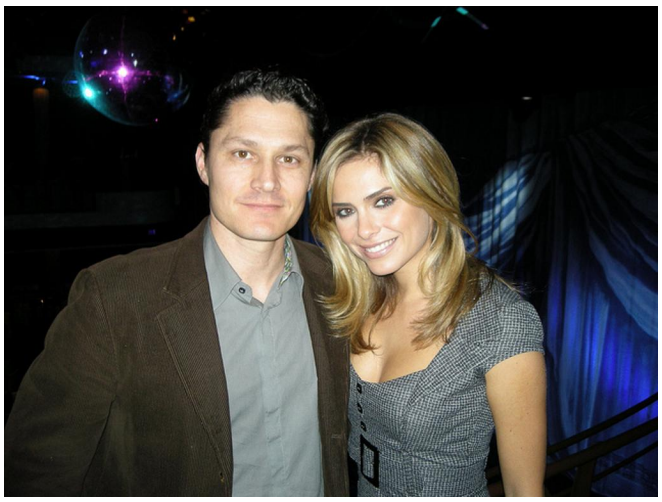
SCRFD-34GF có $34G$ FLOPs nên không thỏa điều kiện. SCRFD-10GF, SCRFD-2.5GF và SCRFD-0.5GF đều thỏa $FLOPs \leq 10G$.

Trong số này, SCRFD-10GF có độ chính xác cao nhất theo bảng đánh giá của InsightFace.

IV.2. AIO25M09FACE02 - EASY

Bạn đang thực hiện huấn luyện mô hình SCRFD dựa vào tài liệu hướng dẫn của `insightface` tại Github. Bạn mong muốn xây dựng một bộ dữ liệu mới để huấn luyện mô hình.

Hãy tải tấm ảnh từ Google Drive và thực hiện gán nhãn khuôn mặt cho ảnh đó. Trong các phương án dưới đây, đâu là cách gán nhãn đúng nhất?



A.
```
# 13_Interview_Interview_2_People_Visible_13_60.jpg
324, 142, 482, 366
516, 232, 674, 422 .
```

B.
```
# 13_Interview_Interview_2_People_Visible_13_60.jpg 1024 768
324 142 482 366
516 232 674 422 .
```

C.
```
# 13_Interview_Interview_2_People_Visible_13_60.jpg 1024 640
516 232 482 366
324 142 674 422 .
```

D.
```
# 13_Interview_Interview_2_People_Visible_13_60.jpg 1024 768
0.3936 0.3307 0.1543 0.2917
0.5811 0.4258 0.1543 0.2474 .
```

Đáp án: B

Giải thích:

A. Đáp án này sai định dạng theo hướng dẫn của InsightFace vì: (1) Thiếu thông tin kích thước ảnh (width và height) sau tên file. (2) Các tọa độ bounding box được phân tách bằng dấu phẩy thay vì khoảng trắng.

B. Đây là đáp án đúng vì: (1) Dòng đầu chứa đúng tên ảnh và kích thước ảnh (1024 768). (2) Các bounding box được ghi đúng thứ tự x_1 y_1 x_2 y_2. (3) Định dạng phân tách bằng khoảng trắng đúng theo chuẩn yêu cầu.

C. Đáp án này sai vì: (1) Kích thước ảnh bị ghi sai (1024 640 thay vì 1024 768). (2) Các bounding box bị gán sai tọa độ, làm thay đổi vị trí và kích thước thực tế của đối tượng.

D. Đáp án này sai vì sử dụng định dạng YOLO (tọa độ chuẩn hóa theo tỉ lệ ảnh) thay vì định dạng tọa độ tuyệt đối x_1 y_1 x_2 y_2 mà InsightFace yêu cầu.

IV.3. AIO25M09FACE03 - MEDIUM

ArcFace (Additive Angular Margin Loss) là hàm mất mát được đề xuất nhằm tăng khả năng phân biệt trong bài toán nhận diện khuôn mặt. Ý tưởng chính là chuẩn hóa (normalize) vector đặc trưng và vector trọng số của lớp về cùng chuẩn L_2, sau đó thêm một *biên góc* (angular margin) m vào góc giữa embedding và lớp đúng để tăng khoảng cách liên lớp và giảm phương sai nội lớp trong không gian đặc trưng.

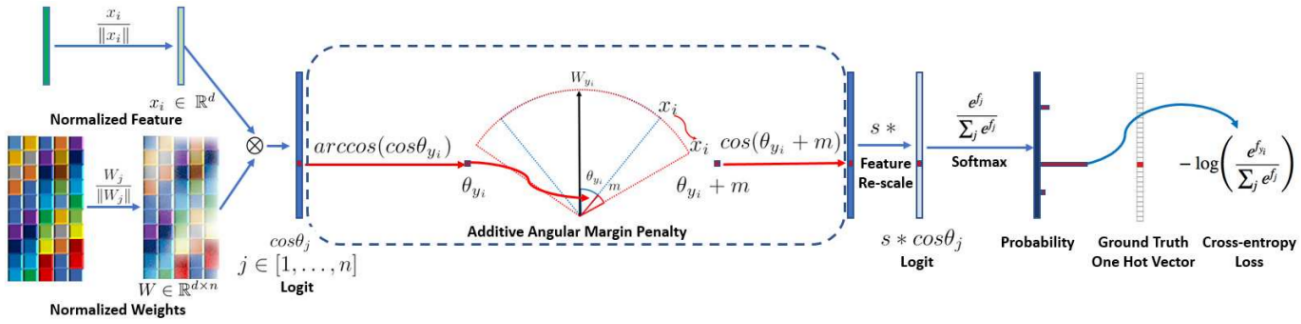
Hàm loss của ArcFace được định nghĩa như sau:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{e^{s\cdot\cos(\theta_{y_i}+m)}}{e^{s\cdot\cos(\theta_{y_i}+m)}+\sum_{j\neq y_i}e^{s\cdot\cos(\theta_j)}}$$

Trong đó:

- N là số mẫu trong batch,
- s là hệ số scale,
- m là angular margin,
- θ_{y_i} là góc giữa embedding của mẫu i và vector trọng số của lớp đúng,
- θ_j là góc giữa embedding và lớp j.

Việc thêm m vào θ_{y_i} giúp tăng độ tách biệt giữa các lớp trên không gian hypersphere.



Giả sử ta thực hiện một bài toán phân loại có 3 lớp. Ta có một mẫu huấn luyện thuộc lớp đúng y. Ta có:

$$\cos\theta_y = 0.8, \quad \cos\theta_2 = 0.3, \quad \cos\theta_3 = -0.2$$

Cho hệ số $s = 30$ và angular margin $m = 0.5$ rad. Giá trị gần đúng của ArcFace loss là:

A. 0.031.

B. 0.31.

C. 1.03.

D. 3.10.

Đáp án: A

Giải thích:

Ta có:

$$\cos\theta_y = 0.8, \quad \cos\theta_2 = 0.3, \quad \cos\theta_3 = -0.2, \quad s = 30, \quad m = 0.5.$$

Dùng máy tính tính trực tiếp:

$$\cos(\theta_y + m) \approx 0.4145.$$

Suy ra các logit:

$$30 \cdot 0.4145 = 12.435, \quad 30 \cdot 0.3 = 9, \quad 30 \cdot (-0.2) = -6.$$

Tính softmax:

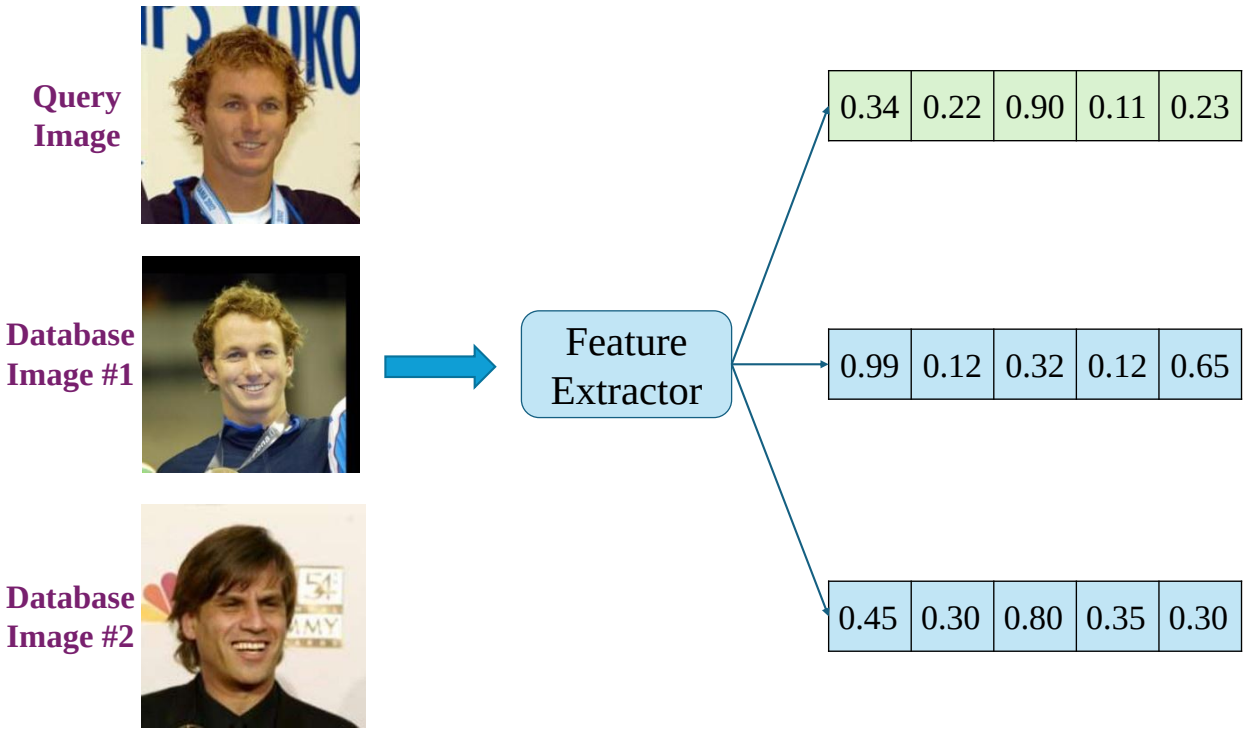
$$P \approx \frac{e^{12.435}}{e^{12.435} + e^{9} + e^{-6}} \approx 0.969.$$

Do đó:

$$L = -\log(0.969) \approx 0.031.$$

IV.4. AIO25M09FACE04 - MEDIUM

Bộ dữ liệu hiện tại gồm 2 ảnh trong cơ sở dữ liệu. Ta đưa 1 ảnh truy vấn và 2 ảnh trong cơ sở dữ liệu vào mô hình Feature Extractor để thu được các vector đặc trưng (embedding) như minh họa phía dưới



Hình 3: Hình ảnh minh họa cho câu AIO2025M09FACE04

Biết rằng ảnh `Image#1` và ảnh truy vấn thực tế thuộc cùng một người (tức là một cặp khớp), còn `Image#2` với ảnh truy vấn thì không. Hãy tính độ tương đồng cosine (cosine similarity) giữa ảnh truy vấn và từng ảnh trong cơ sở dữ liệu, sau đó nhận xét về chất lượng của mô hình.

A. $\cos(Query, Image\#1) \approx 0.376$, $\cos(Query, Image\#2) \approx 0.837$. Mô hình hoạt động tốt.

B. $\cos(Query, Image\#1) \approx 0.376$, $\cos(Query, Image\#2) \approx 0.837$. Mô hình hoạt động chưa tốt.

C. $\cos(Query, Image\#1) \approx 0.645$, $\cos(Query, Image\#2) \approx 0.959$. Mô hình hoạt động tốt.

D. $\cos(Query, Image\#1) \approx 0.645$, $\cos(Query, Image\#2) \approx 0.959$. Mô hình hoạt động chưa tốt.

Đáp án: D

Giải thích: Ta sử dụng công thức cosine similarity:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|}$$

Với:

$$q = [0.34, 0.22, 0.90, 0.11, 0.23],$$

$$d_1 = [0.99, 0.12, 0.32, 0.12, 0.65], \quad d_2 = [0.45, 0.30, 0.80, 0.35, 0.30].$$

Tính tích vô hướng:

$$q \cdot d_1 = 0.8137, \quad q \cdot d_2 = 1.0465.$$

Tính chuẩn:

$$\|q\| \approx 1.0193, \quad \|d_1\| \approx 1.2385, \quad \|d_2\| \approx 1.0700.$$

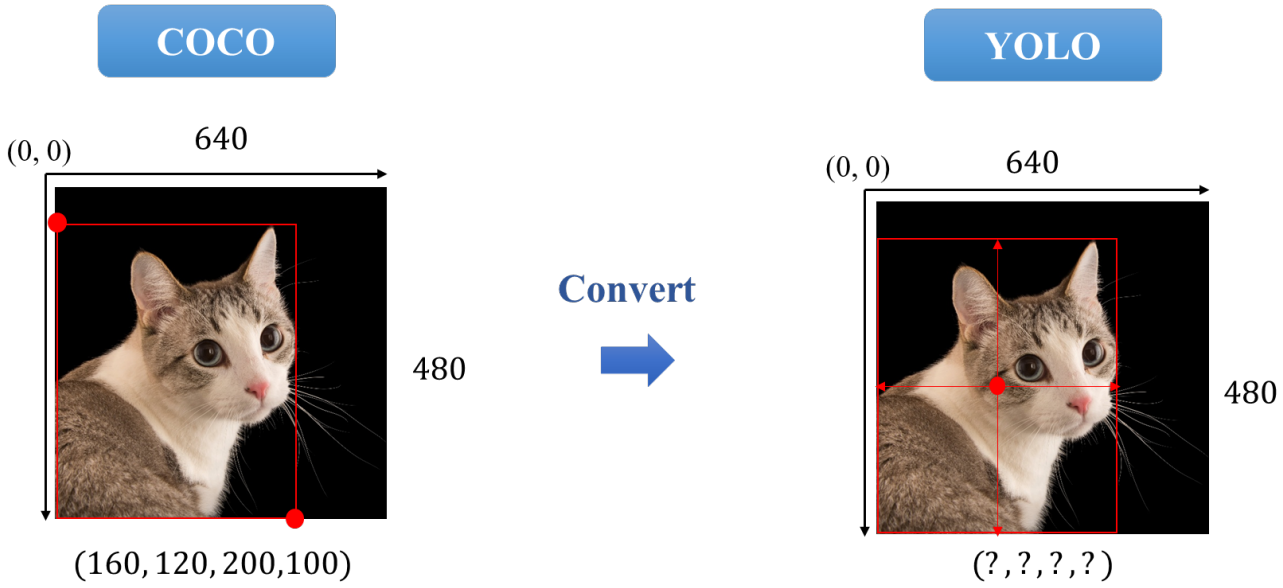
Suy ra:

$$\cos(q, d_1) \approx 0.645, \quad \cos(q, d_2) \approx 0.959.$$

Ta biết rằng hình 2 và ảnh truy vấn không phải là một cặp khớp, nhưng $\cos(q, d_2)$ lại khá cao và cao hơn cả $\cos(q, d_1)$ là ảnh khớp. Do đó, ta kết luận mô hình hoạt động **chưa tốt**.

V. YOLOv1

V.1. AIO25M09YOLO01 - EASY



Tất cả tọa độ được **chuẩn hóa theo kích thước ảnh**, giá trị nào đúng?

A. $x_c = 0.375,\ y_c = 0.3125,\ w = 0.3125,\ h = 0.2083$

B. $x_c = 0.375,\ y_c = 0.3542,\ w = 0.3125,\ h = 0.2083$

C. $x_c = 0.40625,\ y_c = 0.3542,\ w = 0.3125,\ h = 0.2083$

D. $x_c = 0.40625,\ y_c = 0.3125,\ w = 0.3125,\ h = 0.2083$

Đáp án: C

Giải thích:

Ảnh gốc có kích thước:

$$W = 640, \quad H = 480$$

Bounding box trong COCO format:

$$[x_{min}, y_{min}, width, height] = [160, 120, 200, 100]$$

Bước 1: Tính tọa độ tâm của bounding box

Trong YOLO format, cần tọa độ tâm:

$$x_{center} = x_{min} + \frac{width}{2}$$

$$y_{center} = y_{min} + \frac{height}{2}$$

Thay số:

$$x_{center} = 160 + \frac{200}{2} = 160 + 100 = 260$$

$$y_{center} = 120 + \frac{100}{2} = 120 + 50 = 170$$

Bước 2: Chuẩn hóa theo kích thước ảnh

YOLO yêu cầu tất cả tọa độ được chuẩn hóa về khoảng $[0, 1]$ bằng cách chia cho kích thước ảnh.

$$x_c = \frac{x_{center}}{W} = \frac{260}{640} = 0.40625$$

$$y_c = \frac{y_{center}}{H} = \frac{170}{480} \approx 0.3542$$

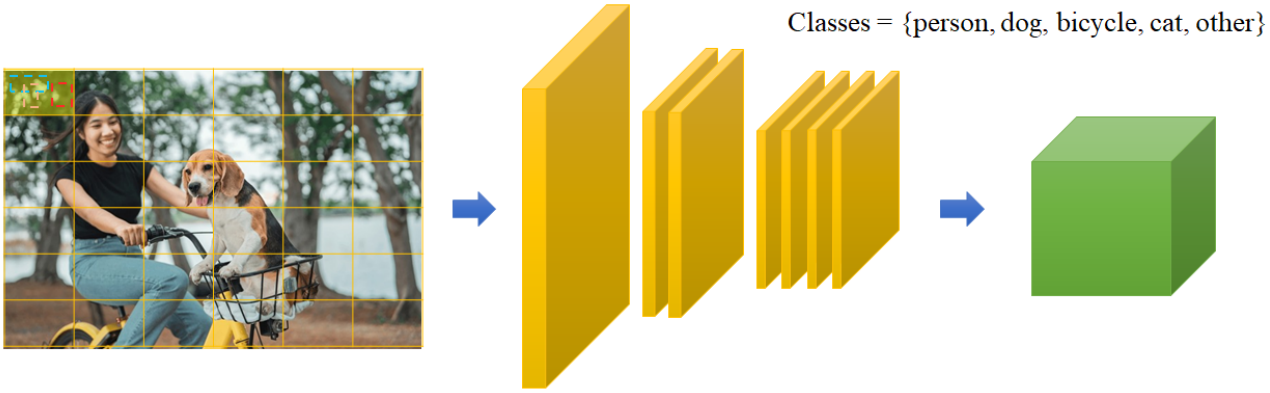
$$w = \frac{width}{W} = \frac{200}{640} = 0.3125$$

$$h = \frac{height}{H} = \frac{100}{480} \approx 0.2083$$

Bước 3: Kết quả YOLO format

$$(x_c, y_c, w, h) = (0.40625,\ 0.3542,\ 0.3125,\ 0.2083)$$

V.2. AIO25M09YOLO02 - EASY



Hình trên minh họa pipeline của **YOLOv1**. Mô hình chia ảnh đầu vào thành lưới $S \times S$. Shape output sau khi flatten của mô hình là bao nhiêu?

A. 468

B. 540

C. 648

D. 720

Đáp án: D

Giải thích:

Bước 1: Số giá trị mỗi grid cell dự đoán

Mỗi bounding box dự đoán:

$$(x, y, w, h, confidence) = 5$$

Với $B = 3$:

$$B \times 5 = 3 \times 5 = 15$$

Ngoài ra mỗi cell dự đoán thêm xác suất cho C lớp:

$$C = 5$$

Do đó tổng số giá trị mỗi cell:

$$15 + 5 = 20$$

Bước 2: Tổng số grid cell

Ảnh được chia thành:

$$S \times S = 6 \times 6 = 36$$

Bước 3: Tổng số phần tử của output tensor

$$36 \times 20 = 720$$

Do đó tensor output có shape:

$$6 \times 6 \times 20$$

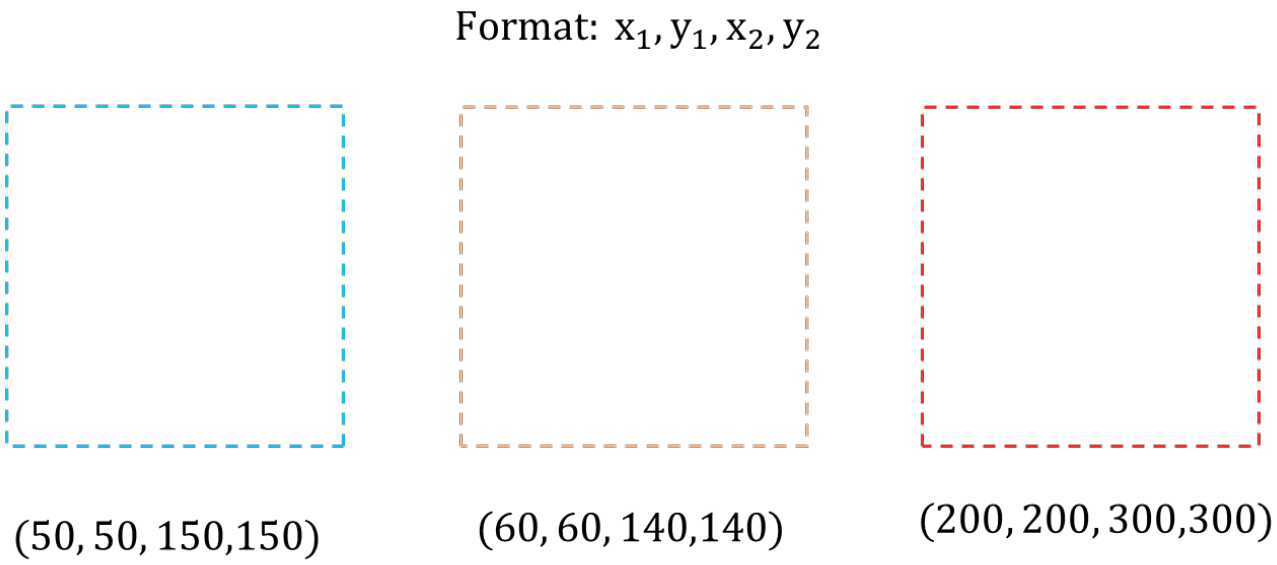
Bước 4: Flatten

Khi flatten toàn bộ tensor:

$$6 \times 6 \times 20 = 720$$

V.3. AIO25M09YOLO03 - EASY

Cho 3 bounding box dự đoán cho cùng một object như hình dưới đây:



$$I_o U_{threshold} = 0.5$$

Sau khi áp dụng **Non-Maximum Suppression (NMS)**, còn lại bao nhiêu bounding box?

A. 1

B. 2

C. 3

D. 0

Đáp án: B

Giải thích: Theo NMS, chúng ta sẽ thực hiện các bước như sau để loại bỏ các bounding box dư thừa **Bước 1: Sắp xếp bounding box theo score giảm dần**

$$B_1(0.9),\ B_2(0.8),\ B_3(0.7)$$

Bước 2: Giữ bounding box có score cao nhất

Giữ lại:

$$B_1$$

Bước 3: Tính IoU giữa B_1 và các box còn lại

IoU(B_1, B_2)

Intersection:

$$x_{left} = 60, \quad y_{top} = 60$$

$$x_{right} = 140, \quad y_{bottom} = 140$$

$$width = 80, \quad height = 80$$

$$Area_{intersection} = 80 \times 80 = 6400$$

Area mỗi box:

$$Area_{B_1} = 100 \times 100 = 10000$$

$$Area_{B_2} = 80 \times 80 = 6400$$

Union:

$$Area_{union} = 10000 + 6400 - 6400 = 10000$$

$$IoU = \frac{6400}{10000} = 0.64$$

$$0.64 > 0.5$$

$\rightarrow B_2$ bị loại.

Bước 4: So sánh B_1 và B_3

Hai box không overlap:

$$IoU(B_1, B_3) = 0$$

$$0 < 0.5$$

$\rightarrow$ giữ lại B_3.

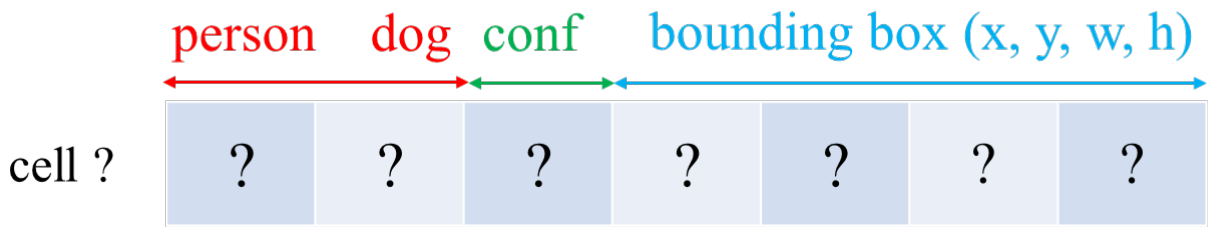
Kết quả sau NMS

Các box còn lại = 2 (B_1, B_3)

V.4. AIO25M09YOLO04 - MEDIUM

Cho một annotation theo định dạng Pascal VOC:

```
<object>
        <name>dog </name>
        <bnbbox>
                <xmin>160</xmin>
                <ymin>120</ymin>
                <xmax>320</xmax>
                <ymax>240</ymax>
        </bnbbox>
</object>
```



Giả sử:

$$Image\ size = 640 \times 480$$

YOLOv1 chia ảnh thành:

$$S = 4 \times 4\ grid$$

Hãy xác định:

- Grid cell nào chịu trách nhiệm dự đoán object
- Giá trị label của cell đó

A. Cell $(1, 1)$; $(0, 1, 1, 0.5, 0.5, 0.25, 0.25)$

B. Cell $(1, 1)$; $(0, 1, 1, 0.5, 0.5, 0.25, 0.1667)$

C. Cell $(2, 1)$; $(0, 1, 1, 0.5, 0.5, 0.25, 0.25)$

D. Cell $(2, 1)$; $(0, 1, 1, 0.5, 0.5, 0.25, 0.1667)$

Đáp án: A

Giải thích

Bước 1: Tính tâm bounding box

$$x_c = \frac{x_{min} + x_{max}}{2}$$

$$y_c = \frac{y_{min} + y_{max}}{2}$$

$$x_c = \frac{160 + 320}{2} = 240$$

$$y_c = \frac{120 + 240}{2} = 180$$

Bước 2: Xác định grid cell chứa tâm

Mỗi cell có kích thước:

$$cell_w = 640/4 = 160$$

$$cell_h = 480/4 = 120$$

Chỉ số cell:

$$col = \lfloor 240/160 \rfloor = 1$$

$$row = \lfloor 180/120 \rfloor = 1$$

→ Object thuộc cell:

$$(1, 1)$$

Bước 3: Tính offset trong cell

Tọa độ góc trên trái cell:

$$x_{cell} = 160$$

$$y_{cell} = 120$$

Offset:

$$x = \frac{240 - 160}{160} = 0.5$$

$$y = \frac{180 - 120}{120} = 0.5$$

Bước 4: Chuẩn hóa width và height

$$w = \frac{320 - 160}{640} = \frac{160}{640} = 0.25$$

$$h = \frac{240 - 120}{480} = \frac{120}{480} = 0.25$$

Bước 5: Label vector

Class = dog

$$(person, dog) = (0, 1)$$

confidence:

$$conf = 1$$

Label cuối cùng

$$(0, 1, 1, 0.5, 0.5, 0.25, 0.25)$$

Cell chịu trách nhiệm:

$$(1, 1)$$

V.5. AIO25M09YOLO05 - MEDIUM

$$\mathcal{L} = \mathcal{L}_{loc} + \mathcal{L}_{obj} + \mathcal{L}_{cls}$$

$$\mathcal{L}_{\text{loc}} = \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} 1_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} 1_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i}\right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i}\right)^2 \right]$$

$$\mathcal{L}_{\text{obj}} = \sum_{i=0}^{S^2} \sum_{j=0}^{B} 1_{ij}^{\text{obj}} \left(C_{ij} - \hat{C}_{ij}\right)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} 1_{ij}^{\text{noobj}} \left(C_{ij} - \hat{C}_{ij}\right)^2$$

$$\mathcal{L}_{\text{cls}} = \sum_{i=0}^{S^2} 1_{i}^{\text{obj}} \sum_{c \in \text{classes}} \left(p_i(c) - \hat{p}_i(c)\right)^2$$

cell predict

person	dog	conf	bounding box (x, y, w, h)			
0.2	0.7	0.8	0.6	0.4	0.3	0.2

Hình 4: Hàm loss tính toàn cho cell predict

Tiếp tục từ kết quả câu **AIO25M09YOLO04**. Giả sử:

$$\lambda_{coord} = 1, \quad \lambda_{noobj} = 1$$

Hãy tính tổng loss của cell này. Chọn giá trị đúng gần nhất:

A. 0.09

B. 0.11

C. 0.15

D. 0.21

Đáp án: D

Giải thích

1. Localization Loss

$$(x - \hat{x})^2 + (y - \hat{y})^2 = (0.5 - 0.6)^2 + (0.5 - 0.4)^2 = 0.01 + 0.01 = 0.02$$

width height (sqrt)

$$(\sqrt{w} - \sqrt{\hat{w}})^2 = (\sqrt{0.25} - \sqrt{0.30})^2 = (0.5 - 0.5477)^2 \approx 0.0023$$

$$(\sqrt{h} - \sqrt{\hat{h}})^2 = (\sqrt{0.25} - \sqrt{0.20})^2 = (0.5 - 0.4472)^2 \approx 0.0028$$

$$L_{loc} = 0.02 + 0.0023 + 0.0028$$

$$L_{loc} \approx 0.0251$$

2. Objectness Loss

$$(C - \hat{C})^2 = (1 - 0.8)^2 = 0.04$$

3. Classification Loss

$$(p - \hat{p})^2$$

person:

$$(0 - 0.2)^2 = 0.04$$

dog:

$$(1 - 0.7)^2 = 0.09$$

$$L_{cls} = 0.13$$

4. Tổng Loss

$$L = L_{loc} + L_{obj} + L_{cls} = 0.0251 + 0.04 + 0.13 = 0.1951 \approx 0.20$$